

MACSE517: Computer Architecture and Organization Laboratory Record

Submitted by :



25MAI0079
Arshed Ayoob P P

To



Dr M Rajasekhara Babu

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

August 2025

INDEX

Week	Date	Concept	Type	S.No	Topic	Page No.
1	10-07-2025				Introduction to OpenMP	5
			Doc	1	Report on “How to create a Project using Visual Studio”	5
			Doc	2	Writing Sample “C” Program to print “Hello World”	8
			Doc	3	How to compile & Execute program	9
			Doc	4	How to create header file and call functions from main	10
			PP	5	Writing a parallel program using OpenMP to print “Hello World”	10
			Doc	6	Setting properties in Visual studio to compile & execute OpenMP program	11
			Doc	7	OpenMP manual link	11
2	17-07-2025	Parallel Construct			Programs using “Parallel Construct (PC)”	12
			PP	8	Writing a parallel program using OpenMP to print “Hello World”	12
			PP		Developing a menu driven program for hello world serial and parallel programs	
3	24-07-2025	Run-time Environment Routines			Programs using “Runtime Environment routines(RER)”	18
			PP		Execution Time using omp_get_wtime()	
			PP	9	Getting thread ID using omp_get_thread_num()	18
			EP	10	Performance Analysis of Serial vs Parallel Execution for hello world program	25
				11	Develop a sample program using Execution of Runtime Environment Routines & develop menu driven program to demonstrate each routine	27
			EP	i.	the omp_set_num_threads routine.	
			EP	ii.	the omp_get_num_threads routine.	

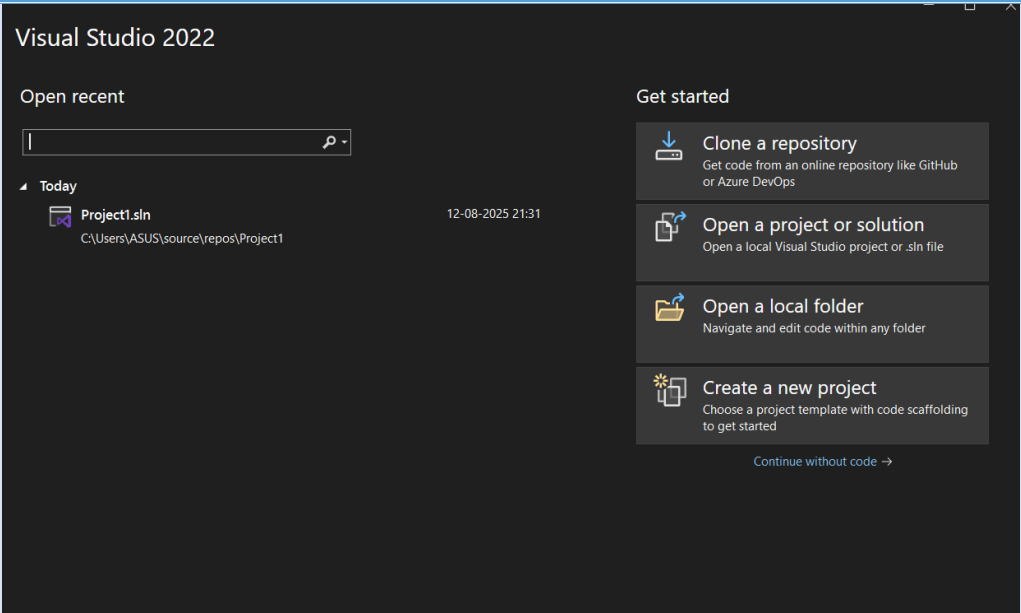
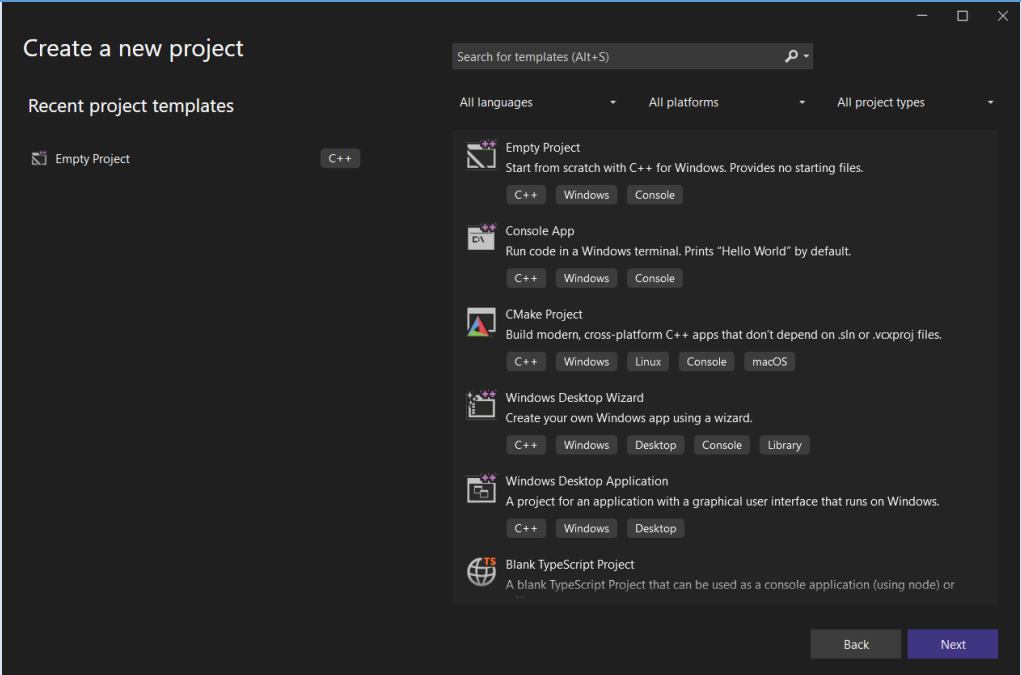
			EP	iii.	the omp_get_max_threads routine.	
			EP	iv.	the omp_get_thread_num routine.	
			EP	v.	the omp_get_num_procs routine.	
			EP	vi.	the omp_in_parallel routine.	
			EP	i.	the omp_set_dynamic routine.	
			EP	ii.	the omp_get_dynamic routine.	
			EP	iii.	the omp_get_cancellation routine	
			EP	iv.	the omp_set_nested routine.	
			EP	v.	the omp_get_nested routine.	
			EP	vi.	the omp_set_schedule routine.	
			EP	vii.	the omp_get_schedule routine.	
			EP	viii.	the omp_get_thread_limit routine.	
			EP	ix.	the omp_set_max_active_levels routine.	
			EP	x.	the omp_get_max_active_levels routine.	
			EP	xi.	the omp_get_level routine.	
			EP	xii.	the omp_get_ancestor_thread_num routine.	
			EP	xiii.	the omp_get_team_size routine.	
			EP	xiv.	the omp_get_active_level routine.	
			EP	xv.	the omp_in_final routine.	
			EP	xvi.	the omp_get_proc_bind routine.	
			EP	xvii.	the omp_set_default_device routine.	
			EP	xviii.	the omp_get_default_device routine.	
			EP	xix.	the omp_get_num_devices routine.	
			EP	xx.	the omp_get_num_teams routine.	
			EP	xxi.	the omp_get_team_num routine.	
		Work-Sharing Constructs			Programs using Loop work sharing construct “for” and Menu driven program	41
4	31-07-2025		PP	12	Two statements and single work-sharing construct sample program	41

			PP	13	Parallelization of square of an array and performance analysis	45
5	07-08-2025				Programs using work sharing construct "Sections"	53
			PP	14	Parallelization of Calculator functions using Sections	53
			EP	15	Parallel Card Game Simulation Using OpenMP Loop, Sections and Single Constructs	53
6	28-08-2025	Scheduling Techniques			Programs using Scheduling techniques	63
			PP	16	Demonstration of Static Scheduling Technique Using OpenMP	63
			PP	17	Demonstration of Dynamic Scheduling Technique	
			PP	18	Demonstration of Guided Scheduling Technique	
			EP	19	Simulating a Bakery Order System Using OpenMP Scheduling Techniques	72
7	04-09-2025	Data Environment Constructs			Programs using Data Environment Constructs	78
			PP	20	Demonstration of Shared & Private Clauses	78
			PP	21	Demonstration of Critical Section	82
			PP	22	Demonstration of Reduction Clause	86
		CUDA			Parallel programs using CUDA	90
8	11-09-2025		Doc		Report on "How to create a Project using GPUs on Google Colab"	90
			Doc		Setting up Jupyter Notebook on Google Colab	91
			Doc		Setting up GPU in Google Colab	92
9	18-09-2025		PP	23	Addition of two vectors using CUDA	93
			PP	24	Matric multiplication using CUDA	

10	25-09-2025		EP	25	Real-Time Traffic Flow Analyzer using OpenMP	101
----	------------	--	----	----	--	-----

Week No: 1	An OpeMP program for printing “hello world” message parallely using all cores				
Date: 10-07-2025					
Objectives	vii.	To demonstrate the utility all cores of CPU			
	viii.	To print hello world message by all cores			
Problem Statement	Report on “How to create a Project using Visual Studio”				
	Writing Sample “C” Program to print “Hello World”				
	How to compile & execute a program				
	How to create a header file and call functions from main				
	Writing a parallel program using OpenMP to print “Hello World”				
	Setting properties in Visual Studio to compile & execute the OpenMP program				
Requirements					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	8
	2	Clock	1.6 GHz	2.5 GHz	3.30 GHz
	3	RAM	4 GB	4GB or more	16 GB
	4	Hard Disk Space	10 GB free	50 + GB free	954 GB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) developed by Microsoft, used for creating software applications, websites, web apps, and mobile applications.	
	2	OpenMP	2.0	OpenMP (Open Multi-Processing) is an API (Application Programming Interface) that enables parallel programming in C, C++, and Fortran. It allows developers to efficiently utilize multiple CPU cores to enhance performance on shared-memory systems.	
	Sl.No	File Name	Description		

List of Files Supplied:	1	helloworld_srl.h	It includes various functions to print "hello world" message using single core as well as multicore	
	1	helloworld_prl.h	Is a menu driven program that is used to navigate across choices of serial functions and parallel functions	
	2	25MAI0079_main.cpp	Used to call parallel and serial function	
List of functions in each file	Sl.No	Function Name	Function Name	Description
	1	helloworld_srl.h	fn_helloworld_srl()	Is function to print a message by using all cores of system in parallel
	2	helloworld_prl.h	fn_helloworld_prl()	Is function to print a message by using all cores of system in Serial
	2	25MAI0079_main.cpp	main()	Used to call the parallel and serial functions.
Program(s)				
helloworld_srl.h	<pre>#include <stdio.h> void fn_helloworld_srl() { printf("Hello World\n"); }</pre>			
helloworld_prl.h	<pre>#include <stdio.h> #include "omp.h" void fn_helloworld_prl() { #pragma omp parallel { printf("Hello World from thread %d\n", omp_get_thread_num()); } }</pre>			
25M9_main.cpp	<pre>#include <stdio.h> #include "helloworld_srl.h" #include "helloworld_prl.h" int main() { fn_helloworld_srl(); fn_helloworld_prl(); }</pre>			

	<pre> return 0; } </pre>		
Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program
	1	#pragma omp parallel	Used in “fn_helloworld_parallel()” function. To parallelize structured block contains messages “Hello world parallel”
Input & Output Snapshots:			
			

Configure your new project

Empty Project C++ Windows Console

Project name

Project1

Location

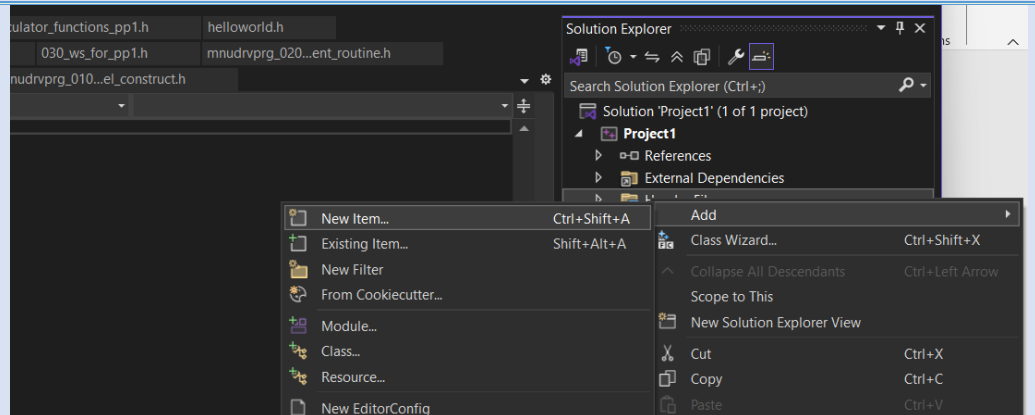
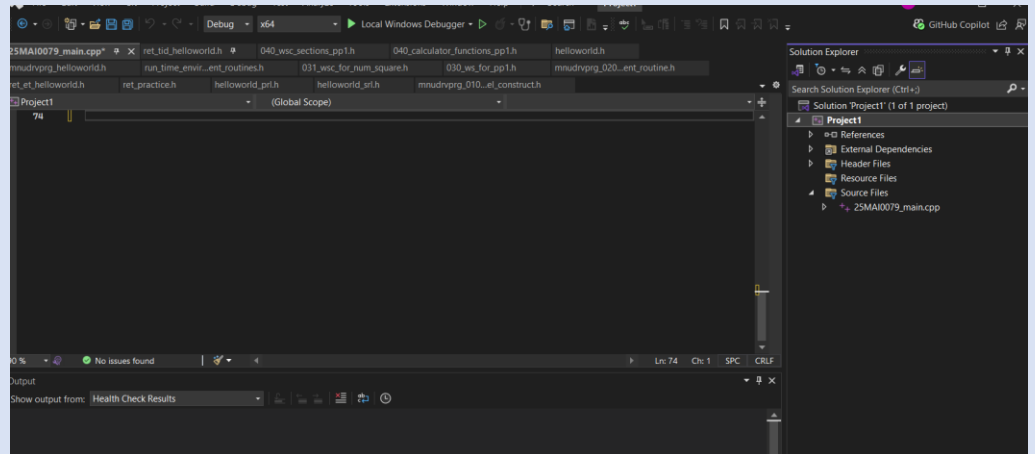
C:\Users\ASUS\source\repos

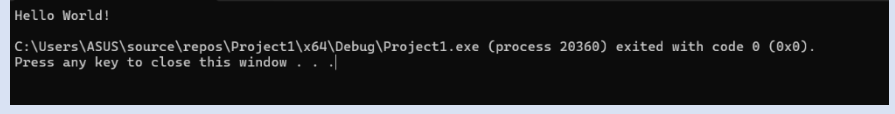
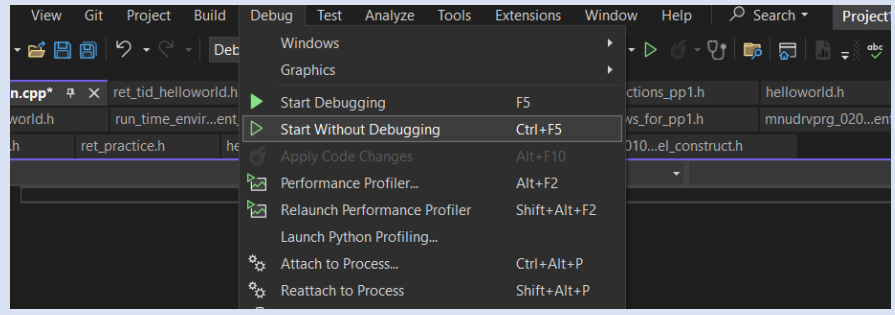
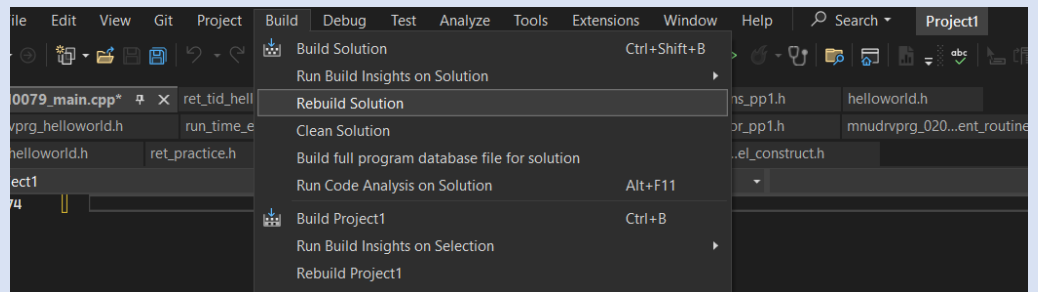
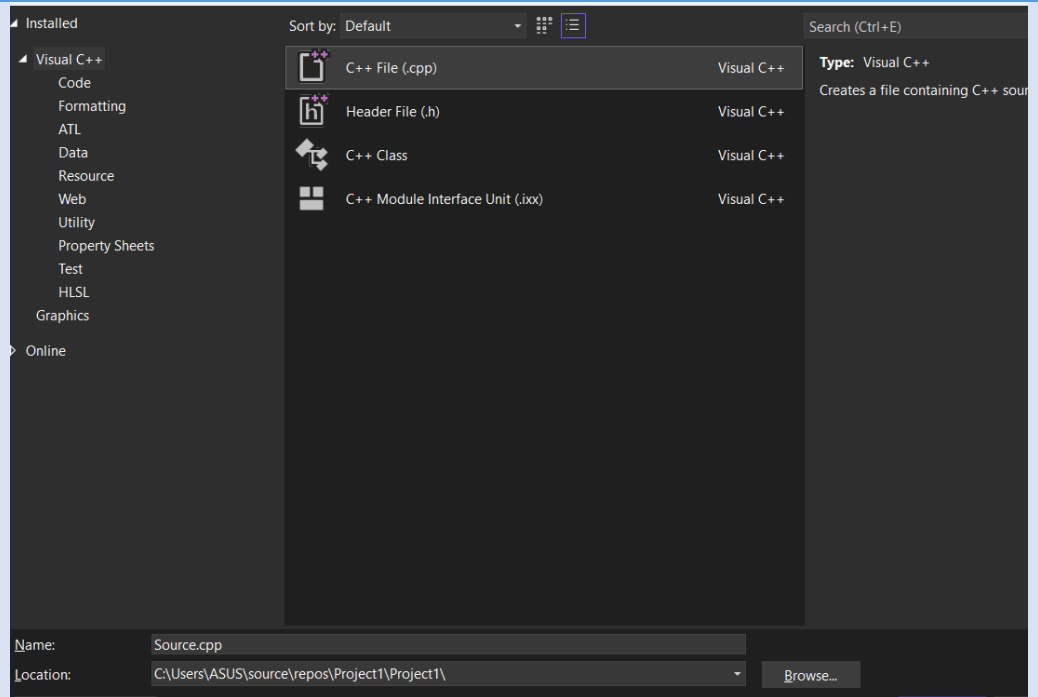
Solution name [?](#)

Project1

☐ Place solution and project in the same directory

Project will be created in "C:\Users\ASUS\source\repos\Project1\Project1"





```

Hello world serial
Hello world parallel
Hello world parallel
Hello world parallel
Hello world parallel
Hello world parallel
Hello world parallel
Hello world parallel
Hello world parallel
Thread ID: 0 (serial execution)
Thread ID: 0
Thread ID: 3
Thread ID: 1
Thread ID: 7
Thread ID: 4
Thread ID: 2
Thread ID: 6
Thread ID: 5

```

C:\Users\ASUS\source\repos\Project1\x64\Debug\Project1.exe (process 7608) exited with code 0 (0x0).
Press any key to close this window . . .|

25MAI0079_main.cpp Property Pages

Configuration: Active(Debug)

Platform: Active(x64)

Configuration Manager...

Configuration Properties

- General
- C/C++
 - General
 - Optimization
 - Preprocessor
 - Code Generation
 - Language
 - Precompiled Headers
 - Output Files
 - Browse Information
 - External Includes
 - Advanced
 - All Options
 - Command Line

Enable String Pooling	No (/Gm-)
Enable Minimal Rebuild	Yes (/EHsc)
Enable C++ Exceptions	No
Smaller Type Check	Both (/RTC1, equiv. to /RTCsu) (/RTC1)
Basic Runtime Checks	Multi-threaded (/MT)
Runtime Library	Multi-threaded (/MT)
Struct Member Alignment	Multi-threaded Debug (/MTd)
Security Check	Multi-threaded DLL (/MD)
Control Flow Guard	Multi-threaded Debug DLL (/MDd)
Enable Function-Level Linking	<inherit from parent or project defaults>
Enable Parallel Code Generation	Not Set
Enable Enhanced Instruction Set	Not Set
Enable Vector Length	Not Set
Floating Point Model	
Enable Floating Point Exceptions	
Create Hotpatchable Image	

25MAI0079_main.cpp Property Pages

Configuration: Active(Debug)

Platform: Active(x64)

Configuration Manager...

Configuration Properties

- General
- C/C++
 - General
 - Optimization
 - Preprocessor
 - Code Generation
 - Language
 - Precompiled Headers
 - Output Files
 - Browse Information
 - External Includes
 - Advanced
 - All Options
 - Command Line

Disable Language Extensions	No
Treat WChar_t As Built in Type	Yes (/Zc:wchar_t)
Force Conformance in For Loop Scope	Yes (/Zc:forScope)
Remove unreferenced code and data	Yes (/Zcinline)
Enforce type conversion rules	
Enable Run-Time Type Information	
Open MP Support	Yes (/openmp)
C++ Language Standard	No (/openmp-)
C Language Standard	Yes (/openmp)
Conformance mode	<inherit from parent or project defaults>
Enable Experimental C++ Standard Library	
Build ISO C++23 Standard Library Modules	

Open MP Support

Enable OpenMP 2.0 language extensions. (/openmp)

Experimental Set-up Details

Utilization	Speed	Base speed:	3.30 GHz
7%	1.28 GHz	Sockets:	1
		Cores:	4
Processes	Threads	Logical processors:	8
254	3855	Virtualization:	Enabled
	Handles	L1 cache:	320 KB
	127324	L2 cache:	5.0 MB
Up time		L3 cache:	12.0 MB
10:01:00:28			

	Device name LAPTOP-VAC9218G Processor 11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz (3.30 GHz) Installed RAM 16.0 GB (15.7 GB usable) Device ID 7ED5E098-3D6C-4B1F-AB04-97C7B11D86CF Product ID 00327-35922-50135-AAOEM System type 64-bit operating system, x64-based processor
Observations	The parallel function demonstrated printing “Hello World Parallel” four times, as shown in Fig. 4, compared to the serial function which printed “Hello World Serial” only once, as shown in Fig. 2. This occurred because the system running the program has four logical processors, as shown in the figure. Overall, the program successfully utilized all CPU cores effectively through parallel execution.
Source code link	Richie21r/25mai0079_CAO: course work :computer architecture and organisation
References	www.openmp.org

Week No: 02		Programs using “Parallel Construct (PC)”				
Date: 06-08-2025						
Sl. No: 08						
Objectives	xxii.	To demonstrate the utility all cores of CPU				
	xxiii.	To print hello world message by all cores				
Problem Statement	To demonstrate the difference between serial and parallel execution in C using OpenMP, a menu-driven program is developed that allows users to choose between two execution modes for displaying a "Hello World" message.					
	The program is designed to: ❖ Implement a serial version of the "Hello World" function that prints the message once in a sequential manner. ❖ Implement a parallel version using appropriate OpenMP directives, where multiple threads print the message concurrently. ❖ Provide a menu-based interface to switch between serial and parallel execution modes, enabling repeated testing and observation. ❖ Showcase the impact of OpenMP constructs on output behavior, highlighting how thread-level parallelism affects message repetition and ordering. ❖ Serve as a foundation for performance analysis , comparing the output and efficiency of parallel execution against the serial approach.					
Requirements						
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system	
	1	Number of Cores	2	4+	8	
	2	Clock	1.6 GHz	2.5 GHz	3.30 GHz	
	3	RAM	4 GB	4GB or more	16 GB	
	4	Hard Disk Space	10 GB free	50 + GB free	954 GB	
Software:	S.No	Software	Version Details	Description		
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) developed by Microsoft. It is used to create software, websites, web apps, and mobile apps.		
	2	OpenMP	2.0	OpenMP (Open Multi-Processing) is an API (Application Programming Interface) that supports parallel programming in C, C++, and Fortran. It allows		

				developers to write code that runs across multiple CPU cores to improve performance on shared-memory systems.
List of Files Supplied:	Sl.No	File Name	Description	
	1	helloworld.h	It includes various functions to print “hello world” message using single core as well as multicore	
	2	mnudrvprg_helloworld.h	Is a menu driven program that is used to navigate across choices of serial functions and parallel functions	
	3	25MAI0079_main.cpp	Used to call a menu driven function	
List of functions in each file	Sl.No	Function Name	Function Name	Description
	1	helloworld.h	fn_helloworld_parallel()	Is function to print a message by using all cores of system in parallel
	2		fn_helloworld_serial()	Is function to print a message by using all cores of system in Serial
	3	mnudrvprg_helloworld.h	fn_mnudrvprg_helloworld()	Is a function that has switch-case control structure for navigating various choices. It includes the following choices 1-Hello world serial program 2-Hello world program parallel
	4	main_regno.cpp	main()	Used to call menu driven function “fn_mnudrvprg_hello world()”
Program(s)				
helloworld.h	<pre>#include<stdio.h> #include<omp.h> void fn_helloworld_parallel()</pre>			

```

{
#pragma omp parallel
    printf("Hello world parallel\n");
}

void fn_helloworld_serial()
{
    printf("Hello world serial \n");
}

```

mnudrvprg_helloworld.
h

```

#pragma once
#include<stdio.h>
#include "helloworld.h"

int fn_mnudrvprg_helloworld()
{
    int choice;
    while (1)
    {
        printf("List of choices\n");
        printf("1-Hello world serial program \t 2-Hello world program parallel\n");

        printf("Enter your choice\n");
        scanf_s("%d", &choice);
        switch (choice)

```

```

{
    case 0:
    {
        printf("You opted to exit current menu\n");
        return 0;
    }
    case 1:
    {
        printf("The following is Hello world serial program output\n");
        fn_helloworld_serial();
        break;
    }
    case 2:
    {

```

```

printf("The following is Hello world parallel program output\n");

fn_helloworld_parallel();

break;

}

default:

{

printf("You opted wrong choice \n");

break;

}

}

}

return 0;

}

```

25MAI0079_main.cpp

```

#include<stdio.h>

#include<omp.h>

#include "mnudrvprg_helloworld.h"

int main()

{

    fn_mnudrvprg_helloworld();

    return 0;

}

```

Description of OpenMP constructs

S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program
1	#pragma omp parallel	Used in “fn_helloworld_parallel()” function. To parallelize structured block contains messages “Hello world parallel”

Input & Output Snapshots:

Figure 1: Choice menu comes up.

```

Menu:
1. Hello World Serial
2. Hello World Parallel
3. Thread ID Serial
4. Thread ID Parallel
0. Exit
Enter your choice: |

```

Figure 2: Printed message “hello world serial” by serial function on selection of choice-1

```
0. Exit
Enter your choice: 1
Hello world serial
```

Figure 3: Printed message “hello world Parallel” by Parallel function on selection of choice-2

```
0. Exit
Enter your choice: 2
Hello world parallel
Hello world parallel
Hello world parallel
Hello world parallel
Hello world parallel
Hello world parallel
Hello world parallel
Hello world parallel
Hello world parallel
```

Figure 4: Printed message “You opted to exit program” exit the program by choice 0;

```
Enter your choice
0
You opted to exit the program.
```

Experimental Set-up Details

Utilization	Speed	Base speed:	3.30 GHz
7%	1.24 GHz	Sockets:	1
Processes	Threads	Cores:	4
251	3544	Logical processors:	8
Up time	Handles	Virtualization:	Enabled
10:01:02:16	126378	L1 cache:	320 KB
		L2 cache:	5.0 MB
		L3 cache:	12.0 MB

Device name	LAPTOP-VAC9218G
Processor	11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz (3.30 GHz)
Installed RAM	16.0 GB (15.7 GB usable)
Device ID	7ED5E098-3D6C-4B1F-AB04-97C7B11D86CF
Product ID	00327-35922-50135-AAOEM
System type	64-bit operating system, x64-based processor

Observations

The parallel function demonstrated printing “Hello World Parallel” four times, as shown in Fig. 4, whereas the serial function printed “Hello World Serial” only once, as shown in Fig. 2.

This behaviour occurred because the system on which the program was executed has four logical processors, as illustrated in the figure.

Ultimately, the program effectively utilized all available CPU cores by leveraging parallel execution.

Source code link	Richie21r/25mai0079_CAO: course work :computer architecture and organisation
References	www.openmp.org

Week No: 3	Programs using “Runtime Environment routines(RER)”				
Date; 24-07-2025 Sl,No: 9,10					
Objectives	1. Execution Time using omp_get_wtime() 2. Getting thread ID using omp_get_thread_num() 3. Performance Analysis of Serial vs Parallel Execution for hello world program				
Problem Statement	Demonstrate the use of OpenMP Runtime Environment Routines (RER) in C programming to analyze thread behavior and measure execution time under parallel and serial contexts. A menu-driven program is to be developed that showcases key OpenMP RER functions and highlights the impact of parallelism on performance and thread management.				
	The program is structured to: ◊ Implement execution time measurement using omp_get_wtime() to compare serial and parallel code segments, providing insights into runtime performance differences. ◊ Retrieve and display the unique thread ID using omp_get_thread_num() inside a parallel region to illustrate how OpenMP handles multiple threads. ◊ Provide a menu interface to select and execute three different functionalities: 1. Measure and compare execution time using omp_get_wtime() 2. Display thread numbers using omp_get_thread_num() 3. Compare serial vs parallel execution of a "Hello world" program ◊ Demonstrate the behavior and output of a serial versus a parallel implementation of the "Hello world" program, emphasizing how thread-level parallelism introduces variations in execution order and output frequency. ◊ Allow repeated and interactive testing through the menu interface to observe changes in behavior under different scenarios or system loads. ◊ Serve as a practical base for understanding the role of OpenMP runtime functions in performance analysis, debugging, and effective use of parallel programming constructs.				
Requirements					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	8
	2	Clock	1.6 GHz	2.5 GHz	3.30 GHz
	3	RAM	4 GB	4GB or more	16 GB
	4	Hard Disk Space	10 GB free	50 + GB free	954 GB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) developed by Microsoft. It is used to create software, websites, web apps, and mobile apps.	
	2	OpenMP	2.0	OpenMP (Open Multi-Processing) is an API	

				(Application Programming Interface) that supports parallel programming in C, C++, and Fortran. It allows developers to write code that runs across multiple CPU cores to improve performance on shared-memory systems.
List of Files Supplied:	Sl.No	File Name	Description	
	1	helloworld.h	It includes various functions to print “hello world” message using single core as well as multicore	
	2	mnudrvprg_helloworld.h	Is a menu driven program that is used to navigate across choices of serial functions and parallel functions	
	3	main_regno.cpp	Used to call a menu driven function	
List of functions in each file	Sl.No	Function Name	Function Name	Description
	1	ret_tid_helloworld.h	fn_ret_tid_helloworld_srl()	A function to print the thread id for serial program.
	2		fn_ret_tid_helloworld_prl()	A function to print the thread id for parallel program.
	3	mnudrvprg_020_runtime_environment_routines.h	fn_mnudrvprg_020_runtime_environment_routines	Is a function that has switch-case control structure for navigating various choices. It include 4 choices from which the user can select.
	4	25MCS0019_MAIN.cpp	main()	Main function is used to call menu driven function “fn_mnudrvprg_helloworld()”
	5	ret_et_helloworld_prl.h	fn_ret_et_helloworld_prl()	To get the execution time of parallel program
	6	ret_et_helloworld_srl.h	fn_ret_et_helloworld_srl()	To get the execution time of serial program
Program(s)				
ret_tid_helloworld.h	<pre>#include"stdio.h" #include"omp.h" void fn_ret_tid_helloworld_srl() {</pre>			

```

        printf("Hello Wolrd Serial printed by thread ID: %d \n", omp_get_thread_num());
    }

    void fn_ret_tid_helloworld_prl() {
        #pragma omp parallel
        printf("Hello World Parallel printed bt thread ID: %d \n", omp_get_thread_num());
    }

```

mnudrvprg_020_runtime_e
nvironment_routines.h

```

#include"stdio.h"
#include"ret_et_helloworld.h"
#include"ret_tid_helloworld.h"

int fn_mnudrvprg_020_runtime_environment_routines()
{
    while (1)
    {
        printf("#####\n");
        printf("List of choices\n");
        printf("0:exit\n");
        printf("1:fn_ret_et_helloworld_srl\n");
        printf("2:fn_ret_et_helloworld_prl\n");
        printf("3:fn_ret_tid_helloworld_srl\n");
        printf("4:fn_ret_tid_helloworld_prl\n");
    }
}

```

```

        printf("#####\n");
        int choice;
        printf("Enter your choice\n");
        scanf_s("%d", &choice);
        if (choice == 0)
        {
            printf("you chose to exit\n");
            break;
        }
        switch (choice)

```

```

{
    case 1:
    {
        printf("This is output of Hello world serial and its execution time\n");
        fn_ret_et_helloworld_srl();
        break;
    }
    case 2:
    {
        printf("This is output of Hello world parallel and its execution
time\n");

        fn_ret_et_helloworld_prl();
        break;
    }

    case 3:
    {
        printf("This is output ID for serial \n");
        fn_ret_tid_helloworld_srl();
        break;
    }
    case 4:
    {
        printf("This is output ID for parallel:\n");
        fn_ret_tid_helloworld_prl();
        break;
    }
    default:
    {
        printf("wrong choice\n");
        break;
    }
}

return 0;
}

```

ret_et_helloworld_prl.h

```
#include <stdio.h>

#include <omp.h>

void fn_ret_et_helloworld_prl()
{
    double start, end;

    start = omp_get_wtime();

    #pragma omp parallel
    {
        printf("This is Hello world parallel program %d\n", omp_get_thread_num());
    }
}
```

```
end = omp_get_wtime();

printf("Execution time of parallel program = %f seconds\n", end - start);
}
```

ret_et_helloworld_srl.h

```
#include <stdio.h>

#include <omp.h>

void fn_ret_et_helloworld_srl()
{
    double start, end;

    start = omp_get_wtime();

    // Serial hello world
    printf("This hello world serial program\n");

    end = omp_get_wtime();

    printf("execution time of serial program = %f\n", end - start);
}
```

25MAI0079_main.cpp

```
#include <stdio.h>
```

```
#include <omp.h>

#include "mnudrvprg_020_runtime_environment_routines.h"

int main() {

fn_mnudrvprg_020_runtime_environment_routines();

return 0;

}
```

Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program
	1	#pragma omp parallel	Used in “fn_helloworld_parallel()” function. To parallelize structured block contains messages “Hello world parallel”

Fig 1: Choice menu comes up.

Input & Output Snapshots:

```
#####
List of choices
#####
0. Exit
1. Execution Time of Serial Programming output
2. Execution Time of Parallel Programming output
3. Thread ID of Serial Programming
4. Thread ID of Parallel Programming
Enter your choice
|
```

Figure 2: Calculates the execution time for serial program

```
Enter your choice
1
Hello World Serial.
Execution time od serial program is 0.000230
```

Figure 3: Calculates the execution time for parallel program

```
Enter your choice
2
Hello World Parallel.
Hello World Parallel.
Hello World Parallel.
Hello World Parallel.
Hello World Parallel.
Hello World Parallel.
Hello World Parallel.
Hello World Parallel.
Hello World Parallel.
Hello World Parallel.
Hello World Parallel.
Hello World Parallel.
Execution time od Parallel program is 0.002165
```

Figure 4: Prints serial thread id

```
Enter your choice
3
Hello world serial printed by Thread ID: 0
```

Figure 5: Prints parallel thread id

```
Enter your choice
4
Hello world parallel printed by Thread ID: 3
Hello world parallel printed by Thread ID: 10
Hello world parallel printed by Thread ID: 9
Hello world parallel printed by Thread ID: 8
Hello world parallel printed by Thread ID: 4
Hello world parallel printed by Thread ID: 5
Hello world parallel printed by Thread ID: 11
Hello world parallel printed by Thread ID: 6
Hello world parallel printed by Thread ID: 1
Hello world parallel printed by Thread ID: 7
Hello world parallel printed by Thread ID: 2
Hello world parallel printed by Thread ID: 0
```

Figure 6: Prints this when you choose exit from the code

```
Enter your choice
0
You opted to exit the program.
```

Experimental Set-up Details

Utilization	Speed	Base speed:	3.30 GHz
7%	1.24 GHz	Sockets:	1
Processes	Threads	Cores:	4
251	3544	Logical processors:	8
Handles		Virtualization:	Enabled
Up time		L1 cache:	320 KB
10:01:02:16		L2 cache:	5.0 MB
		L3 cache:	12.0 MB

Device name	LAPTOP-VAC9218G
Processor	11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz (3.30 GHz)
Installed RAM	16.0 GB (15.7 GB usable)
Device ID	7ED5E098-3D6C-4B1F-AB04-97C7B11D86CF
Product ID	00327-35922-50135-AAOEM
System type	64-bit operating system, x64-based processor

Observations

The program demonstrated the use of OpenMP Runtime Environment Routines to analyze the behavior of serial and parallel execution concerning thread utilization and performance.

The serial function printed the message: "Hello world in serial!", followed by the thread ID (always 0), as shown in Fig 1. It executed the code sequentially, using only a single thread, and measured the execution time using `omp_get_wtime()`.

The parallel function printed: "Hello world in parallel!" multiple times, as shown in Fig 2, because multiple threads were executing concurrently. The number of times the message was printed matched the number of threads spawned, which depends on the number of logical processors available, which in this case was 16. Each thread printed the message independently, and thread IDs ranged from 0 to 15.

The execution time for both serial and parallel versions was measured and displayed in milliseconds using `omp_get_wtime()`. The program then computed:

- Speedup as the ratio of serial time to parallel time
 - Efficiency as the ratio of speedup to the total number of threads, multiplied by 100
- These metrics allowed a comparative analysis of performance. The speedup value indicated how much faster the parallel program was compared to the serial version. The efficiency value showed how effectively the threads were utilized.

Finally, the program demonstrated that parallel execution reduces execution time and enables better utilization of available cores, validating the advantage of using OpenMP for concurrent processing.

Source code link	Richie21r/25mai0079_CAO: course work :computer architecture and organisation
References	www.openmp.org

Week No: 03		Develop a sample program using Execution of Runtime Environment Routines & develop menu-driven program to demonstrate each routine			
Date: 24-07-2025 Sl,No: 11					
Objectives	1.	the omp_get_num_threads routine			
	2.	the omp_set_num_threads routine			
	3.	the omp_get_max_threads routine			
	4.	the omp_get_thread_num routine			
	5.	the omp_get_num_procs routine			
	6.	the omp_in_parallel routine			
	7.	the omp_set_dynamic routine			
	8.	the omp_get_dynamic routine			
	9.	the omp_get_cancellation routine			
	10.	the omp_set_nested routine			
	11.	the omp_get_nested routine			
	12.	the omp_set_schedule routine			
	13.	the omp_get_schedule routine			
	14.	the omp_get_thread_limit routine			
	15.	the omp_set_max_active_levels routine			
	16.	the omp_get_max_active_levels routine			
	17.	the omp_get_level routine			
	18.	the omp_get_ancestor_thread_num routine			
	19.	the omp_get_team_size routine			
	20.	the omp_get_active_level routine			
	21.	the omp_in_final routine			
	22.	the omp_get_proc_bind routine			
	23.	the omp_set_default_device routine			
	24.	the omp_get_default_device routine			
	25.	the omp_get_num_devices routine			
	26.	the omp_get_num_teams routine			
	27.	the omp_get_team_num routine			
	Run these routines using a menu-driven program.				
Problem Statement	Demonstrate the comprehensive use of OpenMP Runtime Environment Routines (RER) in C programming to explore thread management, scheduling policies, device queries, and parallel execution control. A menu-driven program is to be developed that allows interactive execution and observation of multiple OpenMP RER functions, highlighting the behavior of the OpenMP runtime environment under various configurations.				
	Facilitate understanding of low-level OpenMP features, providing a foundational tool for students and developers working on performance tuning, debugging, and exploring advanced features of parallel programming.				
Requirements					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	8
	2	Clock	1.6 GHz	2.5 GHz	3.30 GHz
	3	RAM	4 GB	4GB or more	16 GB
	4	Hard Disk Space	10 GB free	50 + GB free	954 GB

Software:	S.No	Software	Version Details	Description
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) developed by Microsoft. It is used to create software, websites, web apps, and mobile apps.
	2	OpenMP	2.0	OpenMP (Open Multi-Processing) is an API (Application Programming Interface) that supports parallel programming in C, C++, and Fortran. It allows developers to write code that runs across multiple CPU cores to improve performance on shared-memory systems.

List of Files Supplied:	Sl.No	File Name	Description
	1	run_time_environment_routines.h	It consists of 27 run time routines of omp
	2	main_regno.cpp	Used to call a menu driven function

List of functions in each file	Sl.No	Function Name	Function Name	Description
	1	run_time_environment_routines.h	fn_rter_omp_set_num_threads() fn_rter_omp_get_num_threads() fn_rter_omp_get_max_threads() fn_rter_omp_get_thread_num() fn_rter_omp_get_num_procs() fn_rter_omp_in_parallel() fn_rter_omp_set_dynamic() fn_rter_omp_get_dynamic() fn_rter_omp_get_cancellation() fn_rter_omp_set_nested() fn_rter_omp_get_nested() fn_rter_omp_set_schedule()	To run the runtime environment routines of omp.

			fn_rter_omp_get_schedule() fn_rter_omp_get_thread_limit() fn_rter_omp_set_max_active_levels() fn_rter_omp_get_max_active_levels() fn_rter_omp_get_level() fn_rter_omp_get_ancestor_thread_num() fn_rter_omp_get_team_size() fn_rter_omp_get_active_level() fn_rter_omp_in_final() fn_rter_omp_get_proc_bind() fn_rter_omp_set_default_device() fn_rter_omp_get_default_device() fn_rter_omp_get_num_devices() fn_rter_omp_get_num_teams()	
			fn_rter_omp_get_team_num()	
	2	25MAI0079_main.cpp	main()	Used to call a runtime routine functions of omp
Program(s)				
mnudrvprg_helloworld.h	<pre> #include <stdio.h> #include "omp.h" #include <omp_llvm.h> // 1. omp_set_num_threads void fn_rter_omp_set_num_threads(int num_threads) { #pragma omp parallel omp_set_num_threads(num_threads); printf("Number of threads set to: %d\n", num_threads); printf("Thread %d out of %d\n", omp_get_thread_num(), omp_get_num_threads()); </pre>			

```

}

// 2. omp_get_num_threads
void fn_rter_omp_get_num_threads()
{
    int num_threads = omp_get_num_threads();
    printf("Number of threads: %d\n", num_threads);
}

// 3. omp_get_max_threads
void fn_rter_omp_get_max_threads()
{
    int max_threads = omp_get_max_threads();
    printf("Maximum number of threads: %d\n", max_threads);
}

// 4. omp_get_thread_num
void fn_rter_omp_get_thread_num()
{
    int thread_num = omp_get_thread_num();
    printf("Current thread number: %d\n", thread_num);
}

// 5. omp_get_num_procs
void fn_rter_omp_get_num_procs()
{
    int num_procs = omp_get_num_procs();
    printf("Number of processors available: %d\n", num_procs);
}

// 6. omp_in_parallel
void fn_rter_omp_in_parallel()
{
    if (omp_in_parallel())
        printf("Currently in parallel region.\n");
    else

```

```

printf("Not in parallel region.\n");

#pragma omp parallel
{
    if (omp_in_parallel()) {
        printf("Thread %d: Inside parallel region\n", omp_get_thread_num());
    } else {
        printf("Thread %d: Outside parallel region\n", omp_get_thread_num());
    }
}

```

```

}
}

// 7. omp_set_dynamic
void fn_rter_omp_set_dynamic(int dynamic_threads)
{
    #pragma omp parallel
    omp_set_dynamic(dynamic_threads);
    printf("Dynamic threads set to: %d\n", dynamic_threads);
}

// 8. omp_get_dynamic
void fn_rter_omp_get_dynamic()
{
    int dynamic_threads = omp_get_dynamic();
    printf("Dynamic threads: %d\n", dynamic_threads);
}

// 9. omp_get_cancellation
void fn_rter_omp_get_cancellation()
{
    printf("Cancellation is not directly retrievable in OpenMP.\n");
}

// 10. omp_set_nested
void fn_rter_omp_set_nested(int nested)
{
    #pragma omp parallel

```

```

omp_set_nested(nested);

printf("Nested parallelism set to: %d\n", nested);
}

// 11. omp_get_nested
void fn_rter_omp_get_nested()
{
    int nested = omp_get_nested();
    printf("Nested parallelism: %d\n", nested);
}

// 12. omp_set_schedule
void fn_rter_omp_set_schedule(int chunk_size)
{
    #if defined(_OPENMP) && (_OPENMP >= 200805)
        omp_set_schedule(omp_sched_dynamic, chunk_size);
        printf("Schedule set to kind: %d with chunk size: %d\n", omp_sched_dynamic, chunk_size);
    #else
        printf("omp_set_schedule is not supported by this compiler/OpenMP version.\n");
    #endif
}

// 13. omp_get_schedule
void fn_rter_omp_get_schedule()
{
    #if defined(_OPENMP) && (_OPENMP >= 200805)
        int kind, chunk_size;
        omp_get_schedule(&kind, &chunk_size);
        printf("Current schedule kind: %d, chunk size: %d\n", kind, chunk_size);
    #else
        printf("omp_get_schedule is not supported by this compiler/OpenMP version.\n");
    #endif
}

// 14. omp_get_thread_limit
void fn_rter_omp_get_thread_limit()

```

```

{
#ifdef(_OPENMP) && (_OPENMP >= 201107)
    int limit = omp_get_thread_limit();
    printf("Thread limit: %d\n", limit);
#else
    printf("omp_get_thread_limit is not supported by this compiler/OpenMP version.\n");
#endif
}

// 15. omp_set_max_active_levels
void fn_rter_omp_set_max_active_levels(int max_levels)
{
#ifdef(_OPENMP) && (_OPENMP >= 201107)
    omp_set_max_active_levels(max_levels);
    printf("Max active levels set to: %d\n", max_levels);
#else
    printf("omp_set_max_active_levels is not supported by this compiler/OpenMP version.\n");
#endif
}

// 16. omp_get_max_active_levels
void fn_rter_omp_get_max_active_levels()
{
#ifdef(_OPENMP) && (_OPENMP >= 201107)
    int max_levels = omp_get_max_active_levels();
    printf("Max active levels: %d\n", max_levels);
#else
    printf("omp_get_max_active_levels is not supported by this compiler/OpenMP version.\n");
#endif
}

// 17. omp_get_level
void fn_rter_omp_get_level()
{
#ifdef(_OPENMP) && (_OPENMP >= 201107)
    int level = omp_get_level();

```

```

printf("Current parallel level: %d\n", level);

#else

    printf("omp_get_level is not supported by this compiler/OpenMP version.\n");
#endif

}

// 18. omp_get_ancestor_thread_num
void fn_rter_omp_get_ancestor_thread_num(int level)
{
    #if defined(_OPENMP) && (_OPENMP >= 201107)

        int ancestor = omp_get_ancestor_thread_num(level);

        printf("Ancestor thread num at level %d: %d\n", level, ancestor);

    #else

        printf("omp_get_ancestor_thread_num is not supported by this compiler/OpenMP
version.\n");
    #endif

}

// 19. omp_get_team_size
void fn_rter_omp_get_team_size(int level)
{
    #if defined(_OPENMP) && (_OPENMP >= 201107)

```

```

        int size = omp_get_team_size(level);

        printf("Team size at level %d: %d\n", level, size);

    #else

        printf("omp_get_team_size is not supported by this compiler/OpenMP version.\n");
    #endif

}

// 20. omp_get_active_level
void fn_rter_omp_get_active_level()
{
    #if defined(_OPENMP) && (_OPENMP >= 201107)

        int active_level = omp_get_active_level();

        printf("Active parallel level: %d\n", active_level);

    #else

```

```

printf("omp_get_active_level is not supported by this compiler/OpenMP version.\n");
#endif
}

// 21. omp_in_final
void fn_rter_omp_in_final()
{
#ifdef _OPENMP && (_OPENMP >= 201107)
#pragma omp parallel
{
    if (omp_in_final())
        printf("Thread %d: In final region\n", omp_get_thread_num());
    else
        printf("Thread %d: Not in final region\n", omp_get_thread_num());
}
#else
printf("omp_in_final is not supported by this compiler/OpenMP version.\n");
#endif
}

// 22. omp_get_proc_bind
void fn_rter_omp_get_proc_bind()
{
#ifdef _OPENMP && (_OPENMP >= 201307)
    omp_proc_bind_t bind = omp_get_proc_bind();
    printf("Proc bind: %d\n", bind);
#else
printf("omp_get_proc_bind is not supported by this compiler/OpenMP version.\n");
#endif
}

// 23. omp_set_default_device
void fn_rter_omp_set_default_device(int device)
{
#ifdef _OPENMP && (_OPENMP >= 201307)
    omp_set_default_device(device);

```

```

    printf("Default device set to: %d\n", device);

#else

    printf("omp_set_default_device is not supported by this compiler/OpenMP version.\n");
#endif

}

// 24. omp_get_default_device
void fn_rter_omp_get_default_device()
{
#if defined(_OPENMP) && (_OPENMP >= 201307)
    int device = omp_get_default_device();

```

```

    printf("Default device: %d\n", device);

#else

    printf("omp_get_default_device is not supported by this compiler/OpenMP version.\n");
#endif

}

// 25. omp_get_num_devices
void fn_rter_omp_get_num_devices()
{
#if defined(_OPENMP) && (_OPENMP >= 201307)
    int num_devices = omp_get_num_devices();
    printf("Number of devices: %d\n", num_devices);
#else
    printf("omp_get_num_devices is not supported by this compiler/OpenMP version.\n");
#endif
}

// 26. omp_get_num_teams
void fn_rter_omp_get_num_teams()
{
#if defined(_OPENMP) && (_OPENMP >= 201307)
#pragma omp target teams map(from:teams)
    {
        int teams = omp_get_num_teams();
        printf("Number of teams: %d\n", teams);

```

```

    }

    #else

        printf("omp_get_num_teams is not supported by this compiler/OpenMP version.\n");

    #endif

}

// 27. omp_get_team_num

void fn_rter_omp_get_team_num()

{

    #if defined(_OPENMP) && (_OPENMP >= 201307)

        #pragma omp target teams map(from:team_num)

        {

            int team_num = omp_get_team_num();

            printf("Team num: %d\n", team_num);

        }

    #else

        printf("omp_get_team_num is not supported by this compiler/OpenMP version.\n");

    #endif

}

```

25MAI0079_main.cpp

```

#include<stdio.h>

#include<omp.h>

#include "run_time_environment_routines.h"

int main()

{

    fn_rter_omp_set_num_threads(4);

    fn_rter_omp_get_num_threads();

    fn_rter_omp_get_max_threads();

    fn_rter_omp_get_thread_num();

    fn_rter_omp_get_num_procs();

    fn_rter_omp_in_parallel();

```

```
fn_rter_omp_set_dynamic(1);

fn_rter_omp_get_dynamic();

fn_rter_omp_get_cancellation();

fn_rter_omp_set_nested(1);

fn_rter_omp_get_nested();

fn_rter_omp_set_schedule(4);

fn_rter_omp_get_schedule();

fn_rter_omp_get_thread_limit();

fn_rter_omp_set_max_active_levels(2);

fn_rter_omp_get_max_active_levels();

fn_rter_omp_get_level();

fn_rter_omp_get_ancestor_thread_num(0);

fn_rter_omp_get_team_size(0);

fn_rter_omp_get_active_level();

fn_rter_omp_in_final();

fn_rter_omp_get_proc_bind();

fn_rter_omp_set_default_device(0);

fn_rter_omp_get_default_device();
```

```
fn_rter_omp_get_num_devices();

fn_rter_omp_get_num_teams();

fn_rter_omp_get_team_num();

return 0;

}
```

Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program
	1	<code>#pragma omp parallel</code>	Used in “fn_helloworld_parallel()” function. To parallelize structured block contains messages “Hello world parallel”

Input & Output Snapshots:

```
Number of threads set to: 4
Thread 0 out of 1
Number of threads: 1
Maximum number of threads: 12
Current thread number: 0
Number of processors available: 12
Not in parallel region.
Thread 0: Inside parallel region
Thread 1: Inside parallel region
Thread 5: Inside parallel region
Thread 3: Inside parallel region
Thread 11: Inside parallel region
Thread 6: Inside parallel region
Thread 4: Inside parallel region
Thread 8: Inside parallel region
Thread 10: Inside parallel region
Thread 9: Inside parallel region
Thread 7: Inside parallel region
Thread 2: Inside parallel region
Dynamic threads set to: 1
Dynamic threads: 0
Cancellation is not directly retrievable in OpenMP.
Nested parallelism set to: 1
Nested parallelism: 0
omp_set_schedule is not supported by this compiler/OpenMP version.
omp_get_schedule is not supported by this compiler/OpenMP version.
omp_get_thread_limit is not supported by this compiler/OpenMP version.
omp_set_max_active_levels is not supported by this compiler/OpenMP version.
omp_get_max_active_levels is not supported by this compiler/OpenMP version.
omp_get_level is not supported by this compiler/OpenMP version.
omp_get_ancestor_thread_num is not supported by this compiler/OpenMP version.
omp_get_team_size is not supported by this compiler/OpenMP version.
omp_get_active_level is not supported by this compiler/OpenMP version.
omp_in_final is not supported by this compiler/OpenMP version.
omp_get_proc_bind is not supported by this compiler/OpenMP version.
omp_set_default_device is not supported by this compiler/OpenMP version.
omp_get_default_device is not supported by this compiler/OpenMP version.
omp_get_num_devices is not supported by this compiler/OpenMP version.
omp_get_num_teams is not supported by this compiler/OpenMP version.
omp_get_team_num is not supported by this compiler/OpenMP version.
```

Experimental Set-up Details

	Utilization	Speed	Base speed:	3.30 GHz
	7%	1.24 GHz	Sockets:	1
	Processes	Threads	Cores:	4
	251	3544	Logical processors:	8
	Handles	Virtualization: Enabled		
	126378	L1 cache: 320 KB		
	Up time	L2 cache: 5.0 MB		
	10:01:02:16	L3 cache: 12.0 MB		
Observations	Device name	LAPTOP-VAC9218G		
	Processor	11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz (3.30 GHz)		
	Installed RAM	16.0 GB (15.7 GB usable)		
	Device ID	7ED5E098-3D6C-4B1F-AB04-97C7B11D86CF		
	Product ID	00327-35922-50135-AAOEM		
	System type	64-bit operating system, x64-based processor		
Observations	The menu-driven program successfully demonstrated the behavior and output of various OpenMP Runtime Environment Routines (RER), allowing interactive testing of thread management, scheduling, nesting, and processor capabilities. Each selected routine revealed internal details about the OpenMP environment:			
	- For example, <code>omp_get_num_threads()</code> showed the number of threads participating in a parallel region, while <code>omp_get_max_threads()</code> displayed the maximum threads available. <code>omp_get_thread_num()</code> confirmed the unique thread ID assigned to each thread. <code>omp_get_num_procs()</code> verified the number of available logical processors on the system.			
	Running parallel constructs in functions like <code>fn_omp_set_num_threads()</code> led to the spawning of multiple threads. On a system with 16 logical processors, the output showed up to 16 thread executions for parallel sections, confirming full core utilization.			
	This program enabled an in-depth understanding of how OpenMP internally handles threads, levels, scheduling, nesting, and device management — proving to be a valuable tool for learning and performance tuning in parallel programming. Also, all the routines do not work in visual studio 2022. This program contains only those routines that are supported by the compiler.			
Source code link	Richie21r/25mai0079_CAO: course work :computer architecture and organisation			
References	www.openmp.org			

No: 4 Date:31-07-2025 SI,No: 12	Programs using Loop work sharing construct “for” and Menu driven program				
Objectives	Two statements and single work-sharing construct sample program.				
Problem Statement	Design and implement a menu-driven C program that demonstrates the use of OpenMP loop work-sharing construct for to parallelize loop iterations across multiple threads. The program should include: - One function using two separate constructs: #pragma omp parallel followed by #pragma omp for. - Another function using the combined construct: #pragma omp parallel for. This program aims to showcase how OpenMP distributes loop iterations among threads in both styles and helps users understand the difference in syntax and structure while achieving the same parallel execution behavior.				
Requirements					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	8
	2	Clock	1.6 GHz	2.5 GHz	3.30 GHz
	3	RAM	4 GB	4GB or more	16 GB
	4	Hard Disk Space	10 GB free	50 + GB free	954 GB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	Visual Studio 2022 Community Edition	IDE used to write, compile, and run C/C++ programs with OpenMP support.	
	2	OpenMP	OpenMP 2.0	API used to parallelize the program to utilize multiple CPU cores.	
List of Files Supplied:	Sl.No	File Name	Description		
	1	030_wsc_for_pp1.h	Two methods of parallelizing a loop in OpenMP—using separate parallel and for constructs (fn_wsc_for_2constructs) and a combined parallel for construct (fn_wsc_for_1construct)—to distribute iterations of a loop across multiple threads and print thread IDs executing each iteration.		
	3	25MAI0079.cpp	Used to call a functions of the header file.		

List of functions in each file	Sl.No	File Name	Function Name	Description
	1	030_wsc_for_pp1.h	fn_wsc_for_2constructs()	Creates a parallel region and distribute loop iterations among threads, allowing additional code within the parallel block.
	2		fn_wsc_for_1construct()	Creates a parallel region and directly distribute loop iterations among threads in a compact form.
	4	25MAI0079_main.cpp	main()	Used to call the functions that have been defined in the header file.

Program(s)

```
#include"stdio.h"
#include"omp.h"

void fn_wsc_for_2constructs()
{
```

```
    int i;

    #pragma omp parallel
    {
        printf("Thread %d: WORK SHARING STARTS\n", omp_get_thread_num());

        #pragma omp for
        for (i = 0; i < 12; i++)
            printf("Hello World in iteration: %d by thread ID %d\n", i,
                omp_get_thread_num());
    }

    void fn_wsc_for_1construct()
    {
        int i;

        #pragma omp parallel for
        for (i = 0; i < 12; i++)
            printf("Hello World in iteration: %d by thread ID %d\n", i,
                omp_get_thread_num());
    }
```

25MAI0079_main
.cpp

```
#include <stdio.h>

#include <omp.h>

#include "030_wsc_for_pp1.h"

int main() {

    fn_wsc_for_2constructs();

    fn_wsc_for_1construct();

}
```

Description of
OpenMP
constructs

S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program
1	#pragma omp parallel	Parallel Construct — Used in multiple functions (e.g., fn_omp_set_num_threads(), fn_omp_get_num_threads(), etc.) to create a parallel region. This directive enables the structured block following it to be executed by multiple threads concurrently
2	omp_get_thread_n um()	Runtime Environment Routine — Used in fn_omp_get_thread_num() to retrieve the unique ID of a thread in a parallel region

Input & Output
Snapshots:

```
Hello World in iteration: 10 by thread ID 10
Thread 14: WORK SHARING STARTS
Thread 15: WORK SHARING STARTS
Hello World in iteration: 3 by thread ID 3
Thread 6: WORK SHARING STARTS
Hello World in iteration: 7 by thread ID 7
Hello World in iteration: 9 by thread ID 9
Thread 2: WORK SHARING STARTS
Thread 1: WORK SHARING STARTS
Hello World in iteration: 0 by thread ID 0
Thread 4: WORK SHARING STARTS
Hello World in iteration: 4 by thread ID 4
Hello World in iteration: 2 by thread ID 2
Hello World in iteration: 1 by thread ID 1
Hello World in iteration: 6 by thread ID 6
Hello World in iteration: 0 by thread ID 0
Hello World in iteration: 2 by thread ID 2
Hello World in iteration: 1 by thread ID 1
Hello World in iteration: 11 by thread ID 11
Hello World in iteration: 10 by thread ID 10
Hello World in iteration: 3 by thread ID 3
Hello World in iteration: 6 by thread ID 6
Hello World in iteration: 8 by thread ID 8
Hello World in iteration: 4 by thread ID 4
Hello World in iteration: 7 by thread ID 7
Hello World in iteration: 9 by thread ID 9
Hello World in iteration: 5 by thread ID 5
```

Figure 1: Work sharing construct program.

Experimental Set-
up Details

	Utilization	Speed	Base speed:	3.30 GHz
	7%	1.24 GHz	Sockets:	1
	Processes	Threads	Cores:	4
	251	3544	Logical processors:	8
	Handles	Virtualization:	Enabled	
	Up time	L1 cache:	320 KB	
	10:01:02:16	L2 cache:	5.0 MB	
		L3 cache:	12.0 MB	
	Device name	LAPTOP-VAC9218G		
Processor	11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz (3.30 GHz)			
Installed RAM	16.0 GB (15.7 GB usable)			
Device ID	7ED5E098-3D6C-4B1F-AB04-97C7B11D86CF			
Product ID	00327-35922-50135-AAOEM			
System type	64-bit operating system, x64-based processor			
Observations	The program demonstrates work-sharing in OpenMP using two different approaches—separate <code>#pragma omp parallel</code> and <code>#pragma omp for</code> , and the combined <code>#pragma omp parallel for</code> —to parallelize a loop across available threads.			
	In both functions, the loop iterations (from 0 to 11) are distributed among multiple threads, with each thread printing its ID along with the iteration it executes. This confirms that OpenMP efficiently divides the workload, allowing multiple threads to execute loop iterations concurrently, thereby improving performance through parallel execution.			
Source code link				
References	www.openmp.org			

Week No: 4 Date:31-07-2025 Sl,No: 13		Programs using Loop work sharing construct “for” and Menu driven program			
Objectives		Parallelization of square of an array and performance analysis			
Problem Statement		The objective of this program is to demonstrate and compare the performance of serial and parallel execution in computing the square of elements in an array using OpenMP. T			
		It further evaluates the speedup and efficiency achieved by parallel execution using OpenMP's runtime functions.			
		This exercise provides insights into how loop work-sharing constructs enhance performance through multi-threading and highlights the effective utilization of system cores in parallel computation.			
Requirements					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	8
	2	Clock	1.6 GHz	2.5 GHz	3.30 GHz
	3	RAM	4 GB	4GB or more	16 GB
	4	Hard Disk Space	10 GB free	50 + GB free	954 GB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	Visual Studio 2022 Community Edition	IDE used to write, compile, and run C/C++ programs with OpenMP support.	
	2	OpenMP	OpenMP 2.0	API used to parallelize the program to utilize multiple CPU cores.	
List of Files Supplied:	Sl.No	File Name	Description		
	1	031_wsc_for_num_square.h	Two methods of parallelizing a loop in OpenMP—using separate parallel and for constructs (fn_wsc_for_2constructs) and a combined parallel for construct (fn_wsc_for_1construct)—to distribute iterations of a loop across multiple threads and print thread IDs executing each iteration.		
	3	25MAI0079-main.cpp	Used to call a functions of the header file.		
	Sl.No	File Name	Function Name		Description

List of functions in each file	1	030_wsc_for_pp1.h	fn_initializearray()	This function initializes array for the serial program
	2		fn_computeSquares_srl()	This computes squares of the array
	3		fn_displayResult()	This function displays the result
	4		fn_initializearray_prl()	This function initializes for the prallel program
	5		fn_computeSquares_prl()	This function computes squares with openMP
	6		fn_displayResult_prl()	This function displays the result
	7		fn_num_square_srl()	The primary function to calculate square of an array in serial
	8		fn_num_square_prl()	The primary function to calculate square of an array in parallel
	9		fn_num_square_speedup_efficiency()	This function alculates the speedup and efficiency of the program
	10	25MAI0079_main.cpp	main()	Used to call the functions that have been defined in the header file.

Program(s)

030_wsc_for_pp1.h	<pre>#include <stdio.h> #include <omp.h> #define N 15 // Unified size for both serial and parallel parts double Et_start_srl, Et_stop_srl, Et_total_srl; double Et_start_prl, Et_stop_prl, Et_total_prl; double speedup, efficiency;</pre>
	<pre>void fn_initializearray(int a[]) { for (int i = 0; i < N; i++) { a[i] = i + 1; } } void fn_computeSquares_srl(int a[], int square[]) {</pre>

```

printf("\n\n--- Serial Execution ---\n");

for (int i = 0; i < N; i++) {

    square[i] = a[i] * a[i]; // Corrected to actual square

    printf("Square of a[%d] : %d ^ 2 = %d\n", i, a[i], square[i]);

}

}

void fn_displayResult(int square[]) {

    printf("\n\nSquared Result (Serial): \n");

    for (int i = 0; i < N; i++) {

        printf("Squared[%d] = %d\n", i, square[i]);

    }

}

void fn_initializearray_prl(int a[]) {

    for (int i = 0; i < N; i++) {

        a[i] = i + 1;

    }

}

// Parallel: Compute Squares with OpenMP

void fn_computeSquares_prl(int a[], int square[]) {

    printf("\n\n--- Parallel Execution ---\n");

#pragma omp parallel for

    for (int i = 0; i < N; i++) {

        square[i] = a[i] * a[i]; // Corrected to actual square

        printf("Thread %d: a[%d] = %d, Square = %d\n", omp_get_thread_num(), i, a[i], square[i]);

    }

}

void fn_displayResult_prl(int square[]) {

    printf("\n\nSquared Result (Parallel): \n");

#pragma omp parallel for

    for (int i = 0; i < N; i++) {

        printf("Squared[%d] = %d printed by Thread no : %d\n", i, square[i], omp_get_thread_num());

    }

}

```

```

}

int fn_num_square_srl() {
    int a[N], square[N];
    fn_initializearray(a);
    fn_computeSquares_srl(a, square);
    fn_displayResult(square);
    return 0;
}

int fn_num_square_prl() {
    int a[N], square[N];
    fn_initializearray_prl(a);
    fn_computeSquares_prl(a, square);
    fn_displayResult_prl(square);
    return 0;
}

void fn_num_square_speedup_efficiency() {

```

```

    Et_start_srl = omp_get_wtime();
    fn_num_square_srl();
    Et_stop_srl = omp_get_wtime();

    Et_total_srl = Et_stop_srl - Et_start_srl;

    printf("\nTotal time for serial Execution is : %f\n", Et_total_srl);

    Et_start_prl = omp_get_wtime();
    fn_num_square_prl();
    Et_stop_prl = omp_get_wtime();
    Et_total_prl = Et_stop_prl - Et_start_prl;

    printf("\nTotal time for parallel Execution is : %f\n", Et_total_prl);
    speedup = (Et_total_srl / Et_total_prl);
    printf("\nSpeedup is : %f\n", speedup);

    int no_of_cores = omp_get_max_threads();

```

25MAI0079_main
.cpp

```
        efficiency = ((speedup / no_of_cores) * 100);

        printf("Efficiency = %f \n", efficiency);

    }

#include <stdio.h>

#include <omp.h>

#include "031_wsc_for_num_square.h"

int main() {

    int choice;

    do {

        printf("\n===== \n");

        printf("Menu: Number Square using Work-Sharing Construct (OMP)\n");

        printf("===== \n");

        printf("1. Serial Execution\n");

        printf("2. Parallel Execution\n");

        printf("3. Compare Execution Time, Speedup, and Efficiency\n");

        printf("4. Exit\n");

        printf("Enter your choice: ");

        scanf_s("%d", &choice);

        switch (choice) {

            case 1:

                fn_num_square_srl();

                break;

            case 2:

                fn_num_square_prl();

                break;

            case 3:

                fn_num_square_speedup_efficiency();

                break;

            case 4:

                printf("Exiting program...\n");

                break;
```

```

default:

printf("Invalid choice. Please enter 1-4.\n");

}

} while (choice != 4);

return 0;

}

```

Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program
	1	#pragma omp parallel	Parallel Construct — Used in multiple functions (e.g., fn_omp_set_num_threads(), fn_omp_get_num_threads(), etc.) to create a parallel region. This directive enables the structured block following it to be executed by multiple threads concurrently
	2	omp_get_wtime()	Used in the fn_num_square_speedup_efficiency() function to measure the start and end time of serial and parallel executions in order to calculate total execution time, speedup, and efficiency. It returns the elapsed wall clock time in seconds.

Input & Output Snapshots:

```

1. Serial Execution
2. Parallel Execution
3. Compare Execution Time, Speedup, and Efficiency
4. Exit
Enter your choice: 1

--- Serial Execution ---
Square of a[0] : 1 ^ 2 = 1
Square of a[1] : 2 ^ 2 = 4
Square of a[2] : 3 ^ 2 = 9
Square of a[3] : 4 ^ 2 = 16
Square of a[4] : 5 ^ 2 = 25
Square of a[5] : 6 ^ 2 = 36
Square of a[6] : 7 ^ 2 = 49
Square of a[7] : 8 ^ 2 = 64
Square of a[8] : 9 ^ 2 = 81
Square of a[9] : 10 ^ 2 = 100
Square of a[10] : 11 ^ 2 = 121
Square of a[11] : 12 ^ 2 = 144
Square of a[12] : 13 ^ 2 = 169
Square of a[13] : 14 ^ 2 = 196
Square of a[14] : 15 ^ 2 = 225

```

Figure 2: Calculate the square of an array in serial

```
Enter your choice: 2
```

```
--- Parallel Execution ---
Thread 0: a[0] = 1, Square = 1
Thread 2: a[2] = 3, Square = 9
Thread 11: a[11] = 12, Square = 144
Thread 3: a[3] = 4, Square = 16
Thread 7: a[7] = 8, Square = 64
Thread 5: a[5] = 6, Square = 36
Thread 8: a[8] = 9, Square = 81
Thread 4: a[4] = 5, Square = 25
Thread 6: a[6] = 7, Square = 49
Thread 12: a[12] = 13, Square = 169
Thread 10: a[10] = 11, Square = 121
Thread 9: a[9] = 10, Square = 100
Thread 14: a[14] = 15, Square = 225
Thread 1: a[1] = 2, Square = 4
Thread 13: a[13] = 14, Square = 196
```

Figure 2: Calculate the square of an array in parallel

```
Speedup is : 0.816560
Efficiency = 5.103498
```

```
=====
Menu: Number Square using Work-Sharing Construct (OMP)
=====
1. Serial Execution
2. Parallel Execution
3. Compare Execution Time, Speedup, and Efficiency
4. Exit
Enter your choice: 4
Exiting program...
```

Figure 3: Performance analysis by calculating speedup and efficiency

Experimental Set-up Details

Utilization	Speed	Base speed:	3.30 GHz
7%	1.28 GHz	Sockets:	1
		Cores:	4
Processes	Threads	Handles	Logical processors: 8
254	3855	127324	Virtualization: Enabled
Up time		L1 cache:	320 KB
10:01:00:28		L2 cache:	5.0 MB
		L3 cache:	12.0 MB

Device name	LAPTOP-VAC9218G
Processor	11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz (3.30 GHz)
Installed RAM	16.0 GB (15.7 GB usable)
Device ID	7ED5E098-3D6C-4B1F-AB04-97C7B11D86CF
Product ID	00327-35922-50135-AAOEM
System type	64-bit operating system, x64-based processor

Observations	The program effectively illustrates how OpenMP work-sharing constructs can be used to parallelize the computation of array element squares and compares the results with a serial approach. In the serial version, elements are squared one after another, whereas the parallel version allows multiple threads to perform calculations simultaneously, utilizing the system's processor cores. Execution times for both serial and parallel implementations were recorded using <code>omp_get_wtime()</code> to evaluate performance improvement. The results indicate that the parallel approach completes the task faster than the serial one, influenced by the number of logical processors on the system. Additionally, the program computes Speedup (the ratio of serial to parallel execution time) and Efficiency (Speedup divided by the number of threads) to assess how well system resources are utilized. This confirms the advantages of parallelizing computationally intensive tasks and provides insight into OpenMP's runtime behavior.
Source code link	Richie21r/25mai0079_CAO: course work :computer architecture and organisation
References	www.openmp.org

Week No: 6 Date:07-08-2025 S.No: 14	An OpenMP program to show work sharing construct “Sections”				
Objectives:	Parallelizing calculator functions using “Sections”				
Problem Statement	Implement a simple calculator that performs addition and subtraction operations in parallel using OpenMP’s Sections construct. The program demonstrates how multiple independent tasks can be executed concurrently by separate threads to improve performance				
Requirements:					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	6
	2	Clock	1.6 GHz	2.5 GHz	2.6 GHz
	3	RAM	4 GB	4GB or more	8 GB
	4	Hard Disk Space	10 GB free	50 + GB free	1 TB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) from Microsoft that supports multiple programming languages, offering code editing, debugging, and project management tools for efficient software development across platforms.	
	2	OpenMP	2.0	OpenMP 2.0 is an API that supports multi-platform shared-memory parallel programming in C, C++, and Fortran, enabling developers to write parallel code using compiler directives and runtime routines.	
List of Files Supplied:	S.No	File Name	Description		
	1	040_calculator_function_ppl.h	It includes functions to return addition or subtraction of two numbers.		
	2	040_wsc_sections_pp1.h	It includes functions to perform calculator operations both in serial as well as parallel.		
	5	25MAI0079_main.cpp	Used to run the calculator functions		
	S.No	File Name	Function Name	Description	

List of functions in each file	1	040_calculator_function_pp1.h	fn_calculator_add()	Is function to take two numbers and return their sum
	2		fn_calculator_subtract()	Is function to take two numbers and return their difference.
	3	040_wsc_sections_pp1.h	fn_wsc_sections_pp1_srl()	Is function to perform the calculator operations serially.
	4		fn_wsc_sections_pp1_prl()	Is function to perform the calculator operations parallely.
	5	25MAI0079_main.cpp	main()	Used to call the functions "fn_wsc_sections_pp1_srl
				()" and "fn_wsc_sections_pp1_prl()

Program(s)

040_calculator_function_pp1.h	<pre>#include "stdio.h" int fn_calculator_add(int a, int b) { return a + b; } int fn_calculator_subtract(int a, int b) { return a - b; }</pre>
040_wsc_sections_pp1.h	<pre>#include "stdio.h" #include "040_calculator_function_pp1.h" #include "omp.h" void fn_wsc_sections_pp1_srl() { int a = 5, b = 3, sum, difference; double start_time, end_time; start_time = omp_get_wtime(); sum = fn_calculator_add(a, b);</pre>

```

        printf("Addition: %d + %d = %d by thread id: %d\n", a, b, sum,
omp_get_thread_num());

        difference = fn_calculator_subtract(a, b);

        printf("Subtraction: %d - %d = %d by thread id: %d\n", a, b, difference,
omp_get_thread_num());

        end_time = omp_get_wtime();

        printf("Serial execution time: %f seconds\n\n", end_time - start_time);
    }

void fn_wsc_sections_pp1_prl(){
    int a = 5, b = 3, sum, difference;
    double start_time, end_time;
    start_time = omp_get_wtime();

    #pragma omp parallel sections
    {
        #pragma omp section
        {
            sum = fn_calculator_add(a, b);
            printf("Addition: %d + %d = %d by thread id: %d\n", a, b, sum,
omp_get_thread_num());
        }

        #pragma omp section
        {
            difference = fn_calculator_subtract(a, b);
            printf("Subtraction: %d - %d = %d by thread id: %d\n", a, b, difference,
omp_get_thread_num());
        }
    }

    end_time = omp_get_wtime();
    printf("Parallel execution time: %f seconds", end_time - start_time);
}

```

25MAI0079_main
.cpp

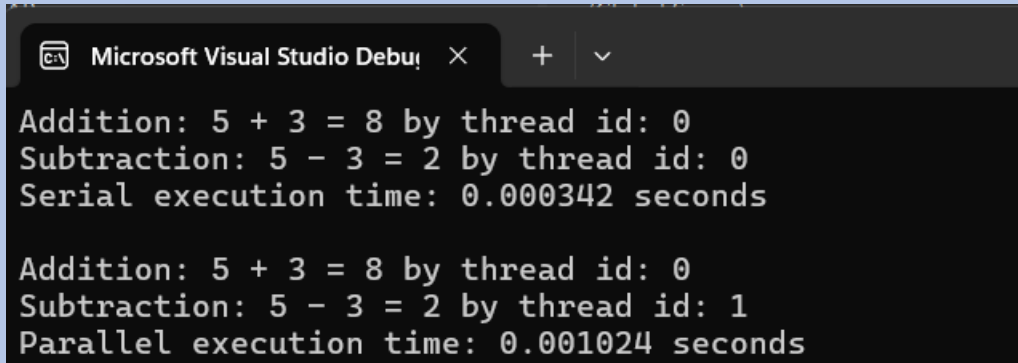
```

#include <stdio.h>
#include <omp.h>
#include "040_wsc_sections_pp1.h"

```

```
int main() {
    fn_wsc_sections_pp1_srl();
    fn_wsc_sections_pp1_prl();
    return 0;
}
```

Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct
	1	#pragma omp parallel sections	Work-Sharing Construct: Used to create a team of threads that execute multiple independent code blocks in parallel. Each section within the construct is assigned to a separate thread, allowing different tasks to be executed concurrently.
	2	#pragma omp section	Work-Sharing Construct: The section directive is used inside a parallel sections block and specifies a block of code that can be executed independently by one thread. Each of them denotes a distinct task that can be run by a separate thread in the team.
	3	omp_get_thread_num()	Runtime Library Routine: Returns unique ID of the calling thread within a team. Used to identify and display which thread is executing which portion of program.

Input & Output Snapshots:	Start Without Debugging:
	 <pre> Addition: 5 + 3 = 8 by thread id: 0 Subtraction: 5 - 3 = 2 by thread id: 0 Serial execution time: 0.000342 seconds Addition: 5 + 3 = 8 by thread id: 0 Subtraction: 5 - 3 = 2 by thread id: 1 Parallel execution time: 0.001024 seconds </pre>

Experimental Set-up Details	Utilization	Speed	Base speed:	3.30 GHz
	7%	1.28 GHz	Sockets:	1
	Processes	Threads	Cores:	4
	254	3855	Logical processors:	8
	Handles		Virtualization:	Enabled
	Up time		L1 cache:	320 KB
	10:01:00:28		L2 cache:	5.0 MB
			L3 cache:	12.0 MB

Observations	ix. The program successfully demonstrates the concept of task-based parallelism using OpenMP's parallel sections construct. x. Independent tasks (addition and subtraction) are assigned to separate threads, allowing them to execute concurrently.
Source code link	Richie21r/25mai0079_CAO: course work :computer architecture and organisation
References	www.openmp.org

Week No: 6 Date:07-08-2025 S.No: 15	An OpenMP Program to Simulate Parallel Card Game Operations				
Objectives:	To demonstrate how OpenMP parallel constructs—for, sections, and single—can be applied to simulate a card game environment involving shuffling, dealing cards, and announcing the start of the game efficiently.				
Problem Statement	Develop an OpenMP C++ program to simulate a simple card game with the following features: xxiv. Shuffling a deck of 52 cards in parallel using a for construct. xxv. Dealing cards concurrently to two players using a sections construct. xxvi. Announcing the start of the game only once using a single construct.				
Requirements:					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	6
	2	Clock	1.6 GHz	2.5 GHz	2.6 GHz
	3	RAM	4 GB	4GB or more	8 GB
	4	Hard Disk Space	10 GB free	50 + GB free	1 TB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) from Microsoft that supports multiple programming languages, offering code editing, debugging, and project management tools for efficient software development across platforms.	
	2	OpenMP	2.0	OpenMP 2.0 is an API that supports multi-platform shared-memory parallel programming in C, C++, and Fortran, enabling developers to write parallel code using compiler directives and runtime routines.	
List of Files Supplied:	S.No	File Name	Description		
	1	040_card_game.h	It includes functions to implement simulation of card game using OpenMP constructs		
	5	25MAI0079_main.cpp	Used to run the card game simulation program		
	S.No	File Name	Function Name		Description

List of functions in each file	1	040_card_game.h	fn_print_hand()	Is function to print the cards dealt to each player
	2		fn_card_game_simul()	Is function to simulate distributing a deck of cards for a game
	3	25MAI0079_main.cpp	main()	Used to call the functions "fn_card_game_simul()"
Program(s)				
	<pre>#include <stdio.h> #include <stdlib.h> #include <omp.h> #include <time.h></pre>			
040_card_game.h	<pre>void fn_print_hand(int* hand, int size, const char* player) { printf("\n%s's hand: ", player); for (int i = 0; i < size; i++) { printf("%d ", hand[i]); } printf("\n"); } void fn_card_game_simul() { int deck[52]; int player1[26], player2[26]; for (int i = 0; i < 52; i++) { deck[i] = i + 1; } srand(time(NULL)); omp_set_num_threads(4); #pragma omp parallel { #pragma omp single</pre>			

```

    {
        printf("Game is starting! Shuffling and dealing cards...\n\n");
    }

#pragma omp for
    for (int i = 0; i < 52; i++) {
        int rand_index = rand() % 52;
        int temp = deck[i];
        deck[i] = deck[rand_index];
        deck[rand_index] = temp;
    }

#pragma omp sections
    {
#pragma omp section
        {
            printf("Player 1 is being dealt cards by Thread %d\n", omp_get_thread_num());
            for (int i = 0; i < 26; i++) {
                player1[i] = deck[i];
            }
        }

#pragma omp section
        {
            printf("Player 2 is being dealt cards by Thread %d\n", omp_get_thread_num());
            for (int i = 26; i < 52; i++) {
                player2[i - 26] = deck[i];
            }
        }
    }

    fn_print_hand(player1, 26, "Player 1");
    fn_print_hand(player2, 26, "Player 2");
}

```

25MAI0079_main.cpp	<pre>#include <stdio.h> #include <omp.h> #include "040_card_game.h" int main() { fn_card_game_simul(); return 0; }</pre>		
Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program
	1	#pragma omp parallel for	Work-Sharing Construct (Loop): Divides deck shuffling among multiple threads in a parallel loop.
	2	#pragma omp sections	Work-Sharing Construct (Sections): Runs distinct card dealing tasks concurrently for two players
	3	#pragma omp single	Work-Sharing Construct (Single): Ensures game start announcement is printed exactly once by one thread.
	4	omp_get_thread_num()	Runtime Library Routine: Returns unique ID of the calling thread within a team. Used to identify and display which thread is executing which portion of program.
Input & Output Snapshots:	Start Without Debugging:		
	Microsoft Visual Studio Debug Console Game is starting! Shuffling and dealing cards... Player 1 is being dealt cards by Thread 0 Player 2 is being dealt cards by Thread 3 Player 1's hand: 24 26 3 11 6 22 1 41 38 12 4 2 42 8 28 29 39 34 32 14 45 35 36 30 49 7 Player 2's hand: 48 51 43 33 47 21 40 44 13 25 9 52 46 37 15 16 17 18 31 20 19 5 27 10 23 50		

Experimental Set-up Details	Utilization	Speed	Base speed:	3.30 GHz	
	7%	1.28 GHz	Sockets:	1	
			Cores:	4	
	Processes	Threads	Handles	Logical processors:	8
	254	3855	127324	Virtualization:	Enabled
	Up time		L1 cache:	320 KB	
	10:01:00:28		L2 cache:	5.0 MB	
			L3 cache:	12.0 MB	
Observations	4. Parallel shuffling improves performance by distributing swap operations among multiple threads.				
	5. Using sections for dealing enables concurrent execution of tasks independent from shuffling.				
	6. The single construct guarantees that only one thread prints the game start message, maintaining clarity.				
	Parallel structures simplify organizing tasks while increasing performance in shared-memory systems.				
Source code link	https://github.com/Shikhar-Tandon15/25MCS0020_CAO_LAB				
References	www.openmp.org				

Week No: 06
Date: 16/10/2025
Sl.No: 1

Demonstrate Scheduling techniques used in OpenMP programming

Objectives

1. To understand how OpenMP handles static scheduling and how iteration distribution among threads changes with different chunk sizes.
2. To understand how OpenMP handles dynamic scheduling and how iteration distribution among threads changes with different chunk sizes.
3. To understand how OpenMP handles **guided scheduling** and how iteration distribution among threads changes with different chunk sizes and uneven workloads.

Problem Statement

1. Schedule(static) Analysis

- **Experiment:** Write an OpenMP program with 4 threads.
 - Test a for loop with schedule(static) for both 16 and 18 iterations.
 - Repeat the tests using schedule(static, 2).
- **Observe & Explain:**
 - How are iterations distributed with and without a chunk size?
 - What happens when the iteration count is not evenly divisible by the thread count?

2. Schedule(dynamic) Analysis

- **Experiment:** Using the same setup, test schedule(dynamic) and schedule(dynamic, 2) for both 16 and 18 iterations.
- **Observe & Explain:**
 - How does the distribution differ from static scheduling?
 - Explain the effect of the chunk size and the behavior with an uneven number of iterations.

3. Schedule(guided) Analysis

- **Experiment:** Using the same setup, test schedule(guided) and schedule(guided, 2) for both 16 and 18 iterations.
- **Observe & Explain:**
 - Describe the pattern of decreasing chunk sizes.
 - How does setting a minimum chunk size affect distribution?

4. Final Comparative Analysis

Based on your experiments, provide a summary comparing static, dynamic, and guided scheduling in terms of:

- **Predictability vs. Overhead:** The trade-offs between pre-determined scheduling and runtime overhead.
- **Load Balancing:** Which strategy is best for uniform vs. non-uniform workloads.
- **Chunk Size:** How chunk size is determined and varies for each type.

Requirements

Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4	8
	2	Clock	1.6 GHz	2.5 GHz	3.30 GHz
	3	RAM	4 GB	8GB	16 GB
	4	Hard Disk Space	10 GB free	20GB	954 GB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) developed by Microsoft. It is used to create software, websites, web apps, and mobile apps.	
	2	OpenMP	2.0	OpenMP (Open Multi-Processing) is an API (Application Programming Interface) that supports parallel programming in C, C++, and Fortran. It allows developers to write code that runs across multiple CPU cores to	

				improve performance on shared-memory systems.
List of Files Supplied:	Sl.No	File Name	Description	
	1	050_Scheduling_Static.h	Demonstrates OpenMP static scheduling with different chunk sizes by showing which thread handles each iteration of a parallel loop.	
	2		Demonstrates OpenMP dynamic scheduling with different chunk sizes, showing how iterations are assigned to threads at runtime.	
	3	050_Scheduling_Guided.h	Demonstrates OpenMP guided scheduling with varying chunk sizes and uneven iterations, showing how threads dynamically take decreasing-sized chunks at runtime.	
	4	25MAI0079_main.cpp	Used to call the static, dynamic, guided scheduling functions.	
List of functions in each file	Sl.No	Function Name	Function Name	Description
	1	050_Scheduling_Static.h	fn_default_Static_Schedule()	Demonstrates OpenMP static scheduling without specifying a chunk size, showing how iterations are evenly divided among threads by default.
			fn_static_01()	Demonstrates OpenMP static scheduling with chunk size 1 , where each thread handles one iteration at a time in a round-robin manner.
			fn_static_02()	Demonstrates OpenMP static scheduling with chunk size 2 , where each thread processes two consecutive iterations before moving to the next.
	2	050_Scheduling_Dynamic.h	fn_default_Dynamic_Schedule()	Demonstrates OpenMP dynamic scheduling without specifying a chunk size, where iterations are assigned to threads as they become available.
			fn_dynamic_01()	Demonstrates OpenMP dynamic scheduling with chunk size 1 , assigning one iteration at a time dynamically to available threads.
			fn_dynamic_02()	Demonstrates OpenMP dynamic scheduling with chunk size 2 , assigning two iterations at a time dynamically to available threads.
		050_Scheduling_Guided.h	fn_default_Guided_Schedule()	Demonstrates OpenMP guided scheduling without specifying a chunk size, where larger chunks are assigned first and decrease over time as threads become available.
			fn_guided_uneven()	Demonstrates guided scheduling with an uneven total iteration

				count (18) to show how OpenMP distributes work dynamically among threads.
			fn_guided_01()	Demonstrates guided scheduling with a minimum chunk size of 1 , ensuring each thread eventually processes at least one iteration at a time.
			fn_guided_02()	Demonstrates guided scheduling with a minimum chunk size of 2 , where OpenMP reduces chunk sizes progressively but keeps them at least two iterations.
	2	25MAI0079_main.cpp	main()	Used to call the static, dynamic, guided scheduling functions.

Program(s)

050_Scheduling_Static.h

```
#include<stdio.h>
#include<omp.h>
void fn_default_Static_Scheduling()
{
    int i;
    int n = 16;
    omp_set_num_threads(4);
    printf("Static Scheduling Example\n");
    #pragma omp parallel for schedule(static)
    for (i = 0;i < n;i++)
    {
        int tid = omp_get_thread_num();
        printf("Iteration %d handled by thread %d\n", i, tid);
    }
}
void fn_static_01()
{
    int i;
    int n = 16;
    omp_set_num_threads(4);
    printf("Static Scheduling Example\n");
    #pragma omp parallel for schedule(static,1)
    for (i = 0;i < n;i++)
    {
        int tid = omp_get_thread_num();
        printf("Iteration %d handled by thread %d\n", i, tid);
    }
}
void fn_static_02()
{
    int i;
    int n = 16;
    omp_set_num_threads(4);
    printf("Static Scheduling Example\n");
    #pragma omp parallel for schedule(static,2)
    for (i = 0;i < n;i++)
    {
        int tid = omp_get_thread_num();
        printf("Iteration %d handled by thread %d\n", i, tid);
    }
}
```

050_Scheduling_Dynamic.h

```
#include<stdio.h>
#include<omp.h>
void fn_default_Dynamic_Scheduling()
{
    int i;
    int n = 16;
    // Set number of threads to 4
    omp_set_num_threads(4);
    printf("Dynamic Scheduling Example:\n");
#pragma omp parallel for schedule(dynamic)
    for (i = 0; i < n; i++) {
        int tid = omp_get_thread_num();
        printf("Iteration %d handled by thread %d\n", i, tid);
    }
}
void fn_dynamic_01()
{
    int i;
    int n = 16;
    // Set number of threads to 4
    omp_set_num_threads(4);
    printf("Dynamic Scheduling Example:\n");
#pragma omp parallel for schedule(dynamic,1)
    for (i = 0; i < n; i++) {
        int tid = omp_get_thread_num();
        printf("Iteration %d handled by thread %d\n", i, tid);
    }
}
void fn_dynamic_02()
{
    int i;
    int n = 16;
    // Set number of threads to 4
    omp_set_num_threads(4);
    printf("Dynamic Scheduling Example:\n");
#pragma omp parallel for schedule(dynamic,2)
    for (i = 0; i < n; i++) {
        int tid = omp_get_thread_num();
        printf("Iteration %d handled by thread %d\n", i, tid);
    }
}
```

050_Scheduling_Guided.h

```
#include<stdio.h>
#include<omp.h>
void fn_default_Guided_Scheduling()
{
    int i;
    int n = 16;
    // Set number of threads to 4
    omp_set_num_threads(4);
    printf("Guided Scheduling Example:\n");
#pragma omp parallel for schedule(guided)
    for (i = 0; i < n; i++) {
        int tid = omp_get_thread_num();
        printf("Iteration %d handled by thread %d\n", i, tid);
    }
}
void fn_guided_uneven()
{
    int i;
    int n = 18;
    // Set number of threads to 4
    omp_set_num_threads(4);
    printf("Guided Scheduling Example:\n");
#pragma omp parallel for schedule(guided)
```

```

    for (i = 0; i < n; i++) {
        int tid = omp_get_thread_num();
        printf("Iteration %d handled by thread %d\n", i, tid);
    }
}

void fn_guided_01()
{
    int i;
    int n = 16;
    // Set number of threads to 4
    omp_set_num_threads(4);
    printf("Guided Scheduling Example:\n");
    #pragma omp parallel for schedule(guided,1)
    for (i = 0; i < n; i++) {
        int tid = omp_get_thread_num();
        printf("Iteration %d handled by thread %d\n", i, tid);
    }
}

void fn_guided_02()
{
    int i;
    int n = 16;

    // Set number of threads to 4
    omp_set_num_threads(4);
    printf("Guided Scheduling Example:\n");
    #pragma omp parallel for schedule(guided,2)
    for (i = 0; i < n; i++) {
        int tid = omp_get_thread_num();
        printf("Iteration %d handled by thread %d\n", i, tid);
    }
}

```

25MAI0079_main.cpp

```

#include <stdio.h>
#include "050_Scheduling_Static.h"
#include "050_Scheduling_Dynamic.h"
#include "050_Scheduling_Guided.h"
int main()
{
    fn_default_Static_Scheduling();
    fn_static_01();
    fn_static_02();
    fn_default_Dynamic_Scheduling();
    fn_dynamic_01();
    fn_dynamic_02();
    fn_default_Guided_Scheduling();
    fn_guided_uneven();
    fn_guided_01();
    fn_guided_02();
    return 0;
}

```

Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program
	1	#pragma omp parallel for scheduling(static)	Divides loop iterations equally among threads before execution , ensuring a fixed workload per thread — used in fn Scheduling_Static.h to demonstrate static scheduling in parallel loops.
	2	#pragma omp parallel for scheduling(dynamic)	Dynamically assigns loop iterations to threads as they become free , balancing workload at runtime — used in fn Scheduling_Dynamic.h to demonstrate dynamic load distribution .

	3	#pragma omp parallel for scheduling(guided)	Distributes iterations in decreasing chunk sizes , starting large and reducing over time — used in fn Scheduling_Guided.h to demonstrate guided adaptive scheduling .
Input & Output Snapshots:			
	1. Default Static scheduling		
	<pre>Static Scheduling Example Iteration 0 handled by thread 0 Iteration 1 handled by thread 0 Iteration 2 handled by thread 0 Iteration 3 handled by thread 0 Iteration 4 handled by thread 1 Iteration 5 handled by thread 1 Iteration 6 handled by thread 1 Iteration 7 handled by thread 1 Iteration 8 handled by thread 2 Iteration 9 handled by thread 2 Iteration 10 handled by thread 2 Iteration 11 handled by thread 2 Iteration 12 handled by thread 3 Iteration 13 handled by thread 3 Iteration 14 handled by thread 3 Iteration 15 handled by thread 3 C:\Users\ASUS\source\repos\practice\x64\Debug\practice.exe (process 12140) exited with code 0 (0x0). Press any key to close this window . . . </pre>		
	2. Static scheduling with chunk size 1		
	<pre>Static Scheduling Example Iteration 0 handled by thread 0 Iteration 4 handled by thread 0 Iteration 8 handled by thread 0 Iteration 12 handled by thread 0 Iteration 3 handled by thread 3 Iteration 7 handled by thread 3 Iteration 11 handled by thread 3 Iteration 15 handled by thread 3 Iteration 1 handled by thread 1 Iteration 5 handled by thread 1 Iteration 9 handled by thread 1 Iteration 13 handled by thread 1 Iteration 2 handled by thread 2 Iteration 6 handled by thread 2 Iteration 10 handled by thread 2 Iteration 14 handled by thread 2 C:\Users\ASUS\source\repos\practice\x64\Debug\practice.exe (process 4260) exited with code 0 (0x0). Press any key to close this window . . . </pre>		
	3. Static scheduling with chunk size 2		
	<pre>Static Scheduling Example Iteration 0 handled by thread 0 Iteration 1 handled by thread 0 Iteration 8 handled by thread 0 Iteration 9 handled by thread 0 Iteration 2 handled by thread 1 Iteration 3 handled by thread 1 Iteration 4 handled by thread 2 Iteration 5 handled by thread 2 Iteration 12 handled by thread 2 Iteration 13 handled by thread 2 Iteration 10 handled by thread 1 Iteration 11 handled by thread 1 Iteration 6 handled by thread 3 Iteration 7 handled by thread 3 Iteration 14 handled by thread 3 Iteration 15 handled by thread 3 C:\Users\ASUS\source\repos\practice\x64\Debug\practice.exe (process 5100) exited with code 0 (0x0). Press any key to close this window . . . </pre>		
	4. Default Dynamic Scheduling		

```
Dynamic Scheduling Example:
Iteration 0 handled by thread 0
Iteration 2 handled by thread 0
Iteration 4 handled by thread 0
Iteration 6 handled by thread 0
Iteration 7 handled by thread 0
Iteration 8 handled by thread 0
Iteration 9 handled by thread 0
Iteration 10 handled by thread 0
Iteration 11 handled by thread 0
Iteration 12 handled by thread 0
Iteration 13 handled by thread 0
Iteration 14 handled by thread 0
Iteration 15 handled by thread 0
Iteration 3 handled by thread 1
Iteration 1 handled by thread 2
Iteration 5 handled by thread 3

C:\Users\ASUS\source\repos\practice\x64\Debug\practice.exe (process 2876) exited with code 0 (0x0).
Press any key to close this window . . .|
```

5. Dynamic Scheduling with chunk size 1

```
Dynamic Scheduling Example:
Iteration 0 handled by thread 0
Iteration 1 handled by thread 1
Iteration 5 handled by thread 1
Iteration 6 handled by thread 1
Iteration 7 handled by thread 1
Iteration 8 handled by thread 1
Iteration 9 handled by thread 1
Iteration 10 handled by thread 1
Iteration 11 handled by thread 1
Iteration 12 handled by thread 1
Iteration 13 handled by thread 1
Iteration 14 handled by thread 1
Iteration 15 handled by thread 1
Iteration 4 handled by thread 3
Iteration 2 handled by thread 2
Iteration 3 handled by thread 0

C:\Users\ASUS\source\repos\practice\x64\Debug\practice.exe (process 2944) exited with code 0 (0x0).
Press any key to close this window . . .|
```

6. Dynamic Scheduling with chunk size 2

```
Dynamic Scheduling Example:
Iteration 0 handled by thread 0
Iteration 6 handled by thread 3
Iteration 7 handled by thread 3
Iteration 8 handled by thread 3
Iteration 9 handled by thread 3
Iteration 10 handled by thread 3
Iteration 11 handled by thread 3
Iteration 12 handled by thread 3
Iteration 13 handled by thread 3
Iteration 14 handled by thread 3
Iteration 15 handled by thread 3
Iteration 4 handled by thread 2
Iteration 5 handled by thread 2
Iteration 2 handled by thread 1
Iteration 3 handled by thread 1
Iteration 1 handled by thread 0

C:\Users\ASUS\source\repos\practice\x64\Debug\practice.exe (process 21240) exited with code 0 (0x0).
Press any key to close this window . . .|
```

7. Default Guided Scheduling with 16 iterations

```
Guided Scheduling Example:
Iteration 0 handled by thread 0
Iteration 1 handled by thread 0
Iteration 7 handled by thread 1
Iteration 8 handled by thread 1
Iteration 9 handled by thread 1
Iteration 12 handled by thread 1
Iteration 13 handled by thread 1
Iteration 14 handled by thread 1
Iteration 15 handled by thread 1
Iteration 2 handled by thread 0
Iteration 3 handled by thread 0
Iteration 10 handled by thread 3
Iteration 11 handled by thread 3
Iteration 4 handled by thread 2
Iteration 5 handled by thread 2
Iteration 6 handled by thread 2

C:\Users\ASUS\source\repos\practice\x64\Debug\practice.exe (process 2044) exited with code 0 (0x0).
Press any key to close this window . . .|
```

8. Default Guided Scheduling with 18 iterations

```
Guided Scheduling Example:
Iteration 0 handled by thread 0
Iteration 1 handled by thread 0
Iteration 2 handled by thread 0
Iteration 3 handled by thread 0
Iteration 4 handled by thread 0
Iteration 14 handled by thread 0
Iteration 12 handled by thread 3
Iteration 13 handled by thread 3
Iteration 16 handled by thread 3
Iteration 17 handled by thread 3
Iteration 9 handled by thread 2
Iteration 10 handled by thread 2
Iteration 11 handled by thread 2
Iteration 5 handled by thread 1
Iteration 6 handled by thread 1
Iteration 7 handled by thread 1
Iteration 8 handled by thread 1
Iteration 15 handled by thread 0

C:\Users\ASUS\source\repos\practice\x64\Debug\practice.exe (process 20912) exited with code 0 (0x0).
Press any key to close this window . . .|
```

9. Guided Scheduling with chunk size 1 (16 iterations)

```
Guided Scheduling Example:
Iteration 0 handled by thread 0
Iteration 1 handled by thread 0
Iteration 2 handled by thread 0
Iteration 3 handled by thread 0
Iteration 12 handled by thread 0
Iteration 13 handled by thread 0
Iteration 14 handled by thread 0
Iteration 15 handled by thread 0
Iteration 7 handled by thread 1
Iteration 8 handled by thread 1
Iteration 9 handled by thread 1
Iteration 10 handled by thread 3
Iteration 11 handled by thread 3
Iteration 4 handled by thread 2
Iteration 5 handled by thread 2
Iteration 6 handled by thread 2

C:\Users\ASUS\source\repos\practice\x64\Debug\practice.exe (process 7360) exited with code 0 (0x0).
Press any key to close this window . . .|
```

10. Guided Scheduling with chunk size 2 (16 iterations)

```
Guided Scheduling Example:
Iteration 0 handled by thread 0
Iteration 1 handled by thread 0
Iteration 2 handled by thread 0
Iteration 10 handled by thread 3
Iteration 11 handled by thread 3
Iteration 12 handled by thread 3
Iteration 13 handled by thread 3
Iteration 14 handled by thread 3
Iteration 15 handled by thread 3
Iteration 4 handled by thread 1
Iteration 5 handled by thread 1
Iteration 6 handled by thread 1
Iteration 7 handled by thread 2
Iteration 8 handled by thread 2
Iteration 9 handled by thread 2
Iteration 3 handled by thread 0

C:\Users\ASUS\source\repos\practice\x64\Debug\practice.exe (process 7556) exited with code 0 (0x0).
Press any key to close this window . . .|
```

Experimental Set-up Details	Utilization	Speed	Base speed:	3.30 GHz
	7%	1.28 GHz	Sockets:	1
	Processes	Threads	Cores:	4
	254	3855	Logical processors:	8
	Up time	Handles	Virtualization:	Enabled
		10:01:00:28	L1 cache:	320 KB
			L2 cache:	5.0 MB
			L3 cache:	12.0 MB
Observations	OpenMP distributes loop iterations differently based on the scheduling type — static scheduling assigns fixed iterations to each thread, dynamic scheduling allocates iterations as threads become available for better load balancing, and guided scheduling starts with larger chunks that decrease over time, efficiently handling uneven workloads .			
Source code link	Richie21r/25mai0079_CAO: course work :computer architecture and organisation			
References	www.openmp.org			

Week No: 6 Date:28-08-2025 S.No: 19	Simulating a Bakery Order System Using OpenMP Scheduling Techniques				
Objectives:	To simulate a bakery model where 20 customer orders, each with varying preparation times, are processed using parallel programming in OpenMP and compare the impact of different scheduling methods on workload distribution and performance.				
Problem Statement	Build an OpenMP-based simulation of a bakery that processes 20 customer orders with varying preparation times using parallel execution. The focus is to analyze how different scheduling clauses (static, dynamic, guided) influence workload distribution among threads and affect total execution time..				
Requirements:					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	6
	2	Clock	1.6 GHz	2.5 GHz	2.6 GHz
	3	RAM	4 GB	4GB or more	8 GB
	4	Hard Disk Space	10 GB free	50 + GB free	1 TB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) from Microsoft that supports multiple programming languages, offering code editing, debugging, and project management tools for efficient software development across platforms.	
	2	OpenMP	2.0	OpenMP 2.0 is an API that supports multi-platform shared-memory parallel programming in C, C++, and Fortran, enabling developers to write parallel code using compiler directives and runtime routines.	
List of Files Supplied:	S.No	File Name	Description		
	1	050_bakery_order_system.h	It includes functions to implement simulation of card game using OpenMP constructs		
	5	25MAI0079_main.cpp	Used to run the card game simulation program		

List of functions in each file	S.No	File Name	Function Name	Description
	1	050_bakery_order_system.h	fn_process_order()	Is function to simulate the processing time
	2		fn_bakery_order_system()	Is function to simulate bakery order processing
	3	25MAI0079_main.cpp	main()	Used to call the functions "fn_bakery_order_system ()"

Program(s)

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
#include <time.h>

#define NUM_ORDERS 20

void fn_process_order(int order_id, int processing_time) {
    for (int i = 0; i < processing_time * 1000000; i++);
    printf("Order %d processed by thread %d with time %d\n",
        order_id, omp_get_thread_num(), processing_time);
}

void fn_bakery_order_system() {
    int orders[NUM_ORDERS];
    int processing_times[NUM_ORDERS];
    double start_time, end_time;

    srand(time(NULL));
    for (int i = 0; i < NUM_ORDERS; i++) {
        orders[i] = i + 1;
        processing_times[i] = (rand() % 5) + 1;
    }

    omp_set_num_threads(4);

    printf("\n=== Static Scheduling ===\n");
```

050_bakery_order_system.h

```

start_time = omp_get_wtime();

#pragma omp parallel for schedule(static)

for (int i = 0; i < NUM_ORDERS; i++) {
    fn_process_order(orders[i], processing_times[i]);
}

end_time = omp_get_wtime();

printf("Static schedule time: %f seconds\n", end_time - start_time);

printf("\n=== Dynamic Scheduling ===\n");

start_time = omp_get_wtime();

#pragma omp parallel for schedule(dynamic)

for (int i = 0; i < NUM_ORDERS; i++) {
    fn_process_order(orders[i], processing_times[i]);
}

end_time = omp_get_wtime();

printf("Dynamic schedule time: %f seconds\n", end_time - start_time);

printf("\n=== Guided Scheduling ===\n");

start_time = omp_get_wtime();

#pragma omp parallel for schedule(guided)

for (int i = 0; i < NUM_ORDERS; i++) {
    fn_process_order(orders[i], processing_times[i]);
}

end_time = omp_get_wtime();

printf("Guided schedule time: %f seconds\n", end_time - start_time);
}

```

25MAI0079_main
.cpp

```

#include <stdio.h>

#include <omp.h>

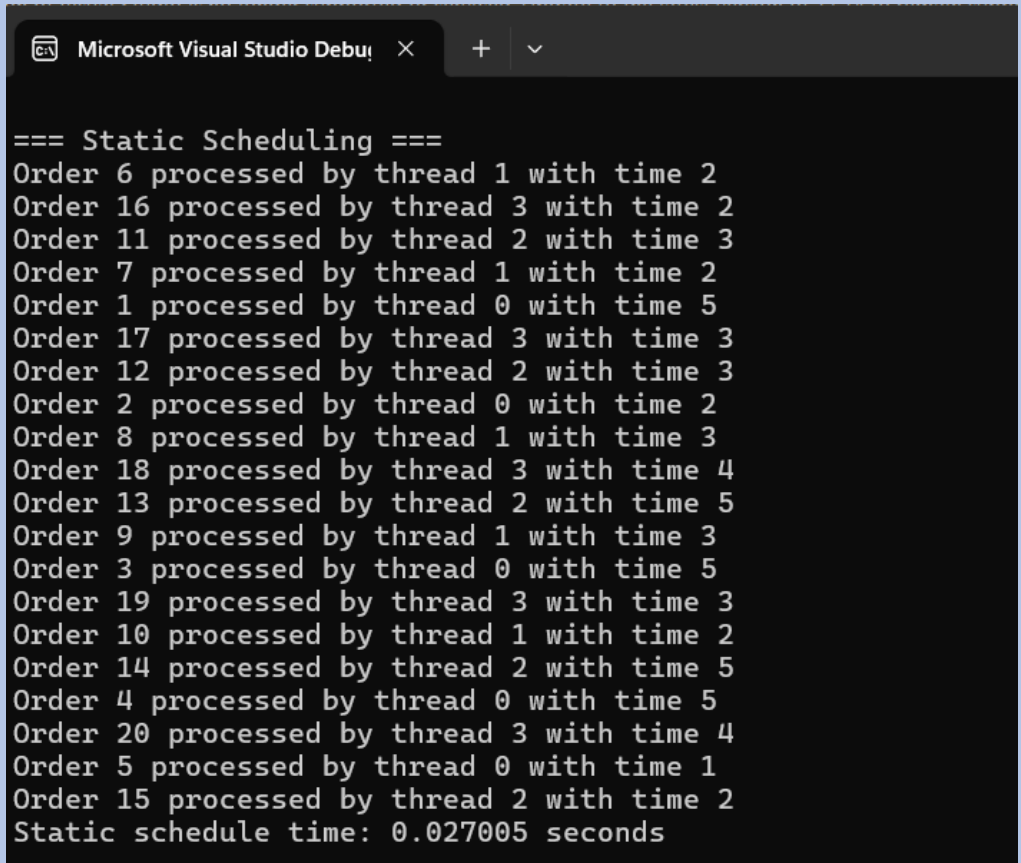
#include "050_bakery_order_system.h"

int main() {
    fn_bakery_order_system();

    return 0;
}

```

Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program
	1	#pragma omp parallel for schedule(static)	Work-Sharing Construct (Loop Scheduling): Distributes loop iterations evenly among threads before execution begins. Each thread gets a fixed contiguous block of iterations determined at compile time.
	2	#pragma omp parallel for schedule (dynamic)	Work-Sharing Construct (Loop Scheduling): Iterations are assigned to threads dynamically during runtime. When a thread finishes its assigned work, it requests the next available chunk.
	5	#pragma omp parallel for schedule (guided)	Work-Sharing Construct (Loop Scheduling): Iterations are dynamically assigned in exponentially decreasing chunk sizes. Initially, large chunks are assigned to threads; as execution progresses, smaller chunks are given.
	4	omp_get_wtime()	Runtime Library Routine: Returns elapsed wall-clock time in seconds.

Input & Output Snapshots:	Start Without Debugging:
	 <pre> === Static Scheduling === Order 6 processed by thread 1 with time 2 Order 16 processed by thread 3 with time 2 Order 11 processed by thread 2 with time 3 Order 7 processed by thread 1 with time 2 Order 1 processed by thread 0 with time 5 Order 17 processed by thread 3 with time 3 Order 12 processed by thread 2 with time 3 Order 2 processed by thread 0 with time 2 Order 8 processed by thread 1 with time 3 Order 18 processed by thread 3 with time 4 Order 13 processed by thread 2 with time 5 Order 9 processed by thread 1 with time 3 Order 3 processed by thread 0 with time 5 Order 19 processed by thread 3 with time 3 Order 10 processed by thread 1 with time 2 Order 14 processed by thread 2 with time 5 Order 4 processed by thread 0 with time 5 Order 20 processed by thread 3 with time 4 Order 5 processed by thread 0 with time 1 Order 15 processed by thread 2 with time 2 Static schedule time: 0.027005 seconds </pre>

```

Microsoft Visual Studio Debug Console
=== Dynamic Scheduling ===
Order 2 processed by thread 2 with time 2
Order 5 processed by thread 2 with time 1
Order 4 processed by thread 1 with time 5
Order 3 processed by thread 0 with time 5
Order 1 processed by thread 3 with time 5
Order 6 processed by thread 2 with time 2
Order 7 processed by thread 1 with time 2
Order 10 processed by thread 2 with time 2
Order 8 processed by thread 0 with time 3
Order 9 processed by thread 3 with time 3
Order 11 processed by thread 1 with time 3
Order 12 processed by thread 2 with time 3
Order 15 processed by thread 1 with time 2
Order 16 processed by thread 2 with time 2
Order 13 processed by thread 0 with time 5
Order 14 processed by thread 3 with time 5
Order 17 processed by thread 1 with time 3
Order 19 processed by thread 0 with time 3
Order 18 processed by thread 2 with time 4
Order 20 processed by thread 3 with time 4
Dynamic schedule time: 0.025458 seconds

```

```

=== Guided Scheduling ===
Order 6 processed by thread 2 with time 2
Order 10 processed by thread 1 with time 2
Order 7 processed by thread 2 with time 2
Order 13 processed by thread 3 with time 5
Order 11 processed by thread 1 with time 3
Order 1 processed by thread 0 with time 5
Order 2 processed by thread 0 with time 2
Order 8 processed by thread 2 with time 3
Order 12 processed by thread 1 with time 3
Order 14 processed by thread 3 with time 5
Order 15 processed by thread 1 with time 2
Order 9 processed by thread 2 with time 3
Order 16 processed by thread 1 with time 2
Order 3 processed by thread 0 with time 5
Order 17 processed by thread 3 with time 3
Order 18 processed by thread 2 with time 4
Order 19 processed by thread 1 with time 3
Order 20 processed by thread 3 with time 4
Order 4 processed by thread 0 with time 5
Order 5 processed by thread 0 with time 1
Guided schedule time: 0.027429 seconds

```

Experimental Set-up Details

Utilization	Speed	Base speed:	3.30 GHz
7%	1.28 GHz	Sockets:	1
		Cores:	4
Processes	Threads	Handles	Logical processors: 8
254	3855	127324	Virtualization: Enabled
Up time			L1 cache: 320 KB
10:01:00:28			L2 cache: 5.0 MB
			L3 cache: 12.0 MB

Observations	xi. Static Scheduling: Fastest when order preparation times are uniform; poor load distribution for unpredictable workloads. xii. Dynamic Scheduling: Provides better balancing for irregular tasks, but with higher runtime overhead. xiii. Guided Scheduling: Starts with larger chunks (for faster threads) and reduces size, giving balance and reducing overhead. xiv. For input with random processing times, Guided Scheduling typically offers the best performance-to-overhead ratio.
Source code link	Richie21r/25mai0079_CAO: course work :computer architecture and organisation
References	www.openmp.org

Week No: 7 Date:04-09-2025 S.No: 20	Parallel Vector Addition Using Shared and Private Clauses				
Objectives:	To demonstrate the use of private and shared OpenMP clauses in a parallel region by performing vector addition. The experiment shows how shared data can be accessed by all threads while private data is specific to each thread.				
Problem Statement	Develop an OpenMP-based program to perform vector addition in parallel. Use the private and shared clauses to handle data efficiently and avoid data conflicts. Display how parallel threads handle individual elements using thread-local copies of variables..				
Requirements:					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	6
	2	Clock	1.6 GHz	2.5 GHz	2.6 GHz
	3	RAM	4 GB	4GB or more	8 GB
	4	Hard Disk Space	10 GB free	50 + GB free	1 TB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) from Microsoft that supports multiple programming languages, offering code editing, debugging, and project management tools for efficient software development across platforms.	
	2	OpenMP	2.0	OpenMP 2.0 is an API that supports multi-platform shared-memory parallel programming in C, C++, and Fortran, enabling developers to write parallel code using compiler directives and runtime routines.	
List of Files Supplied:	S.No	File Name	Description		
	1	060_Data_env_constrcuts.h	It includes functions to implement parallelising of vector addition using OpenMP shared and private class		
	2	25MAI0079_main.cpp	Used to run the vector addition program		
	S.No	File Name	Function Name		Description

List of functions in each file	1	060_Data_env_constructs.h	fn_work()	Performs vector addition using parallel for with private and shared
			fn_run_vector_addition()	Initializes arrays, calls fn_work(), and displays result data
	2	25MAI0079_main.cpp	main()	Used to call the function "fn_run_vector_addition()"
Program(s)				
	<pre>#include <stdio.h> #include <stdlib.h> #include <omp.h> void fn_work(float* a, float* b, float* c, int N) {</pre>			
	<pre> float x, y; int i; #pragma omp parallel for private(x, y) shared(a, b, c) for (i = 0; i < N; i++) { x = a[i]; y = b[i]; c[i] = x + y; } } void fn_run_vector_addition() { int N = 10; float a[10], b[10], c[10]; for (int i = 0; i < N; i++) { a[i] = i * 1.0; b[i] = (N - i) * 1.0; } fn_work(a, b, c, N);</pre>			

060_Data_env_constructs.h

```
printf("Result of vector addition:\n");  
for (int i = 0; i < N; i++) {  
    printf("c[%d] = %.2f\n", i, c[i]);  
}  
}
```

25MAI0079_main.cpp

```
#include <stdio.h>  
#include <omp.h>  
#include "060_Data_env_constructs.h"  
  
int main() {  
    fn_run_vector_addition();  
    return 0;  
}
```

Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program
	1	private(x, y)	Data Environment Clause: Declares variables x and y as private to each thread
	2	shared(a, b, c)	Data Environment Clause: Array data is shared among all threads
	3	#pragma omp parallel for	Work-Sharing Construct: Distributes loop iterations across multiple threads.

Input & Output Snapshots:

Start Without Debugging:

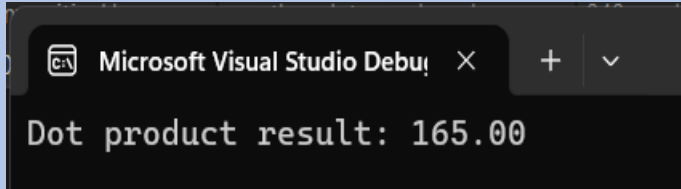


```
Microsoft Visual Studio Debug Console  
Result of vector addition:  
c[0] = 10.00  
c[1] = 10.00  
c[2] = 10.00  
c[3] = 10.00  
c[4] = 10.00  
c[5] = 10.00  
c[6] = 10.00  
c[7] = 10.00  
c[8] = 10.00  
c[9] = 10.00
```

Experimental Set-up Details	Utilization	Speed	Base speed:	3.30 GHz
	7%	1.28 GHz	Sockets:	1
	Processes	Threads	Cores:	4
	254	3855	Logical processors:	8
	Handles	Virtualization:	Enabled	
	127324	L1 cache:	320 KB	
	Up time	L2 cache:	5.0 MB	
10:01:00:28	L3 cache:	12.0 MB		
Observations	xxvii.	Arrays a, b, and c were shared across threads.		
	xxviii.	Temporary variables x and y were made private to avoid race conditions.		
	xxix.	Each iteration was handled in parallel without data corruption.		
Source code link	Richie21r/25mai0079_CAO: course work :computer architecture and organisation			
References	www.openmp.org			

Week No: 7 Date:04-09-2025 S.No: 21	Parallel Dot Product of Two Vectors Using Critical Section in OpenMP				
Objectives:	To demonstrate the use of the critical clause in OpenMP to safely update a shared variable in a parallel region. This experiment shows how to compute the dot product of two vectors in parallel while preventing race conditions using critical sections.				
Problem Statement	Write an OpenMP program that computes the dot product of two vectors in parallel. During the computation, multiple threads attempt to update a shared variable (sum). Use the critical directive to avoid race conditions and ensure the correct result.				
Requirements:					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	6
	2	Clock	1.6 GHz	2.5 GHz	2.6 GHz
	3	RAM	4 GB	4GB or more	8 GB
	4	Hard Disk Space	10 GB free	50 + GB free	1 TB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) from Microsoft that supports multiple programming languages, offering code editing, debugging, and project management tools for efficient software development across platforms.	
	2	OpenMP	2.0	OpenMP 2.0 is an API that supports multi-platform shared-memory parallel programming in C, C++, and Fortran, enabling developers to write parallel code using compiler directives and runtime routines.	
List of Files Supplied:	S.No	File Name	Description		
	1	060_Data_env_critical.h	It includes functions to implement parallelising of dot product of two vectors using OpenMP critical section construct.		
	2	25MAI0079_main.cpp	Used to run the vector dot product program		
	S.No	File Name	Function Name		Description

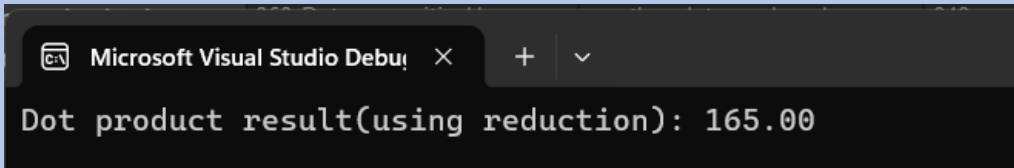
List of functions in each file	1	060_Data_env_critical.h	fn_dot_prod()	Computes the dot product of two arrays using critical to safely update the result
			fn_run_dot_prod()	Initializes vectors, calls fn_dot_prod() function, and displays the result
	2	25MAI0079_main.cpp	main()	Used to call the function "fn_run_dot_prod()"
Program(s)				
	<pre>#include <stdio.h> #include <stdlib.h> #include <omp.h></pre>			
060_Data_env_critical.h	<pre>float fn_dot_prod(float* a, float* b, int N) { float sum = 0.0; #pragma omp parallel for for (int i = 0; i < N; i++) { #pragma omp critical sum += a[i] * b[i]; } return sum; } void fn_run_dot_prod() { int N = 10; float a[10], b[10], result; for (int i = 0; i < N; i++) { a[i] = i * 1.0; b[i] = (N - i) * 1.0; } result = fn_dot_prod(a, b, N);</pre>			

	<pre>printf("Dot product result: %.2f\n", result); }</pre>																																		
25MAI0079_main.cpp	<pre>#include <stdio.h> #include <omp.h> #include "060_Data_env_critical.h" int main() { fn_run_dot_prod(); return 0; }</pre>																																		
Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program																																
	1	#pragma omp parallel for	Work-Sharing Construct: Distributes loop iterations across multiple threads																																
	2	#pragma omp critical	Synchronization Construct: Ensures that only one thread updates the shared variable at a time																																
Input & Output Snapshots:	Start Without Debugging: 																																		
Experimental Set-up Details	<table> <tr> <td>Utilization</td><td>Speed</td><td>Base speed:</td><td>3.30 GHz</td></tr> <tr> <td>7%</td><td>1.28 GHz</td><td>Sockets:</td><td>1</td></tr> <tr> <td>Processes</td><td>Threads</td><td>Cores:</td><td>4</td></tr> <tr> <td>254</td><td>3855</td><td>Logical processors:</td><td>8</td></tr> <tr> <td>Handles</td><td></td><td>Virtualization:</td><td>Enabled</td></tr> <tr> <td>127324</td><td></td><td>L1 cache:</td><td>320 KB</td></tr> <tr> <td>Up time</td><td></td><td>L2 cache:</td><td>5.0 MB</td></tr> <tr> <td>10:01:00:28</td><td></td><td>L3 cache:</td><td>12.0 MB</td></tr> </table>			Utilization	Speed	Base speed:	3.30 GHz	7%	1.28 GHz	Sockets:	1	Processes	Threads	Cores:	4	254	3855	Logical processors:	8	Handles		Virtualization:	Enabled	127324		L1 cache:	320 KB	Up time		L2 cache:	5.0 MB	10:01:00:28		L3 cache:	12.0 MB
Utilization	Speed	Base speed:	3.30 GHz																																
7%	1.28 GHz	Sockets:	1																																
Processes	Threads	Cores:	4																																
254	3855	Logical processors:	8																																
Handles		Virtualization:	Enabled																																
127324		L1 cache:	320 KB																																
Up time		L2 cache:	5.0 MB																																
10:01:00:28		L3 cache:	12.0 MB																																
Observations	7. The sum variable is shared among all threads. Activating #pragma omp critical ensures that at any time, only one thread updates it.																																		

- | | |
|------------------|--|
| | <ol style="list-style-type: none">8. Removing the critical section here will cause race conditions and yield incorrect results.9. Use of critical can slow down performance if the section is large or the thread count is high, but it is necessary for correctness where shared updates happen. |
| | |
| Source code link | Richie21r/25mai0079_CAO: course work :computer architecture and organisation |
| References | www.openmp.org |

Week No: 7 Date:04-09-2025 S.No: 22	Parallel Dot Product Using Reduction Clause in OpenMP				
Objectives:	To illustrate the use of the reduction clause in OpenMP which enables safe and efficient parallel computation of a dot product without race conditions or explicit synchronization.				
Problem Statement	Write an OpenMP-based program to compute the dot product of two vectors in parallel. Use the reduction clause to handle the accumulation of the result and prevent race conditions while maintaining performance.				
Requirements:					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	6
	2	Clock	1.6 GHz	2.5 GHz	2.6 GHz
	3	RAM	4 GB	4GB or more	8 GB
	4	Hard Disk Space	10 GB free	50 + GB free	1 TB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) from Microsoft that supports multiple programming languages, offering code editing, debugging, and project management tools for efficient software development across platforms.	
	2	OpenMP	2.0	OpenMP 2.0 is an API that supports multi-platform shared-memory parallel programming in C, C++, and Fortran, enabling developers to write parallel code using compiler directives and runtime routines.	
List of Files Supplied:	S.No	File Name	Description		
	1	060_Data_env_critical.h	It includes functions to implement parallelising of dot product of two vectors using OpenMP reduction clause.		
	2	25MCS0020_main.cpp	Used to run the vector dot product program		

List of functions in each file	S.No	File Name	Function Name	Description
	1	060_Data_env_reduction.h	fn_calc_dot_prod()	Initializes arrays, calculates dot product using reduction, and prints results
	2	25MAI0079_main.cpp	main()	Used to call the function "fn_calc_dot_prod()"
Program(s)				
060_Data_env_reduction.h	<pre>#include <stdio.h> #include <omp.h> void fn_calc_dot_prod() { int N = 10; float a[10], b[10], sum = 0.0; for (int i = 0; i < N; i++) {</pre>			
	<pre> a[i] = i * 1.0; b[i] = (N - i) * 1.0; } #pragma omp parallel for reduction(+:sum) for (int i = 0; i < N; i++) { sum += a[i] * b[i]; } printf("Dot product result(using reduction): %.2f\n", sum); }</pre>			
25MAI0079_main.cpp	<pre>#include <stdio.h> #include <omp.h> #include "060_Data_env_reduction.h"</pre>			

	<pre>int main() { fn_calc_dot_prod(); return 0; }</pre>																																		
Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program																																
	1	#pragma omp parallel for	Work-Sharing Construct: Distributes loop iterations across multiple threads																																
	2	reduction(+:sum)	Data Environment Clause(Reduction Construct): Each thread performs accumulation in private variable sum and combines safely																																
Input & Output Snapshots:	Start Without Debugging:																																		
																																			
Experimental Set-up Details	<table><tr><td>Utilization</td><td>Speed</td><td>Base speed:</td><td>3.30 GHz</td></tr><tr><td>7%</td><td>1.28 GHz</td><td>Sockets:</td><td>1</td></tr><tr><td>Processes</td><td>Threads</td><td>Cores:</td><td>4</td></tr><tr><td>254</td><td>3855</td><td>Logical processors:</td><td>8</td></tr><tr><td>Handles</td><td>Virtualization:</td><td>L1 cache:</td><td>Enabled</td></tr><tr><td>127324</td><td></td><td>L2 cache:</td><td>320 KB</td></tr><tr><td>Up time</td><td></td><td>L3 cache:</td><td>5.0 MB</td></tr><tr><td>10:01:00:28</td><td></td><td></td><td>12.0 MB</td></tr></table>			Utilization	Speed	Base speed:	3.30 GHz	7%	1.28 GHz	Sockets:	1	Processes	Threads	Cores:	4	254	3855	Logical processors:	8	Handles	Virtualization:	L1 cache:	Enabled	127324		L2 cache:	320 KB	Up time		L3 cache:	5.0 MB	10:01:00:28			12.0 MB
	Utilization	Speed	Base speed:	3.30 GHz																															
7%	1.28 GHz	Sockets:	1																																
Processes	Threads	Cores:	4																																
254	3855	Logical processors:	8																																
Handles	Virtualization:	L1 cache:	Enabled																																
127324		L2 cache:	320 KB																																
Up time		L3 cache:	5.0 MB																																
10:01:00:28			12.0 MB																																
Observations	• The reduction clause enables transparent parallel accumulation, eliminating the need for critical or atomic constructs. • Execution is efficient as each thread computes its local partial sum, later combined to produce the correct global result.																																		

	• Suitable for operations like summation, product, min, max where associative operators are used.
Source code link	Richie21r/25mai0079_CAO: course work :computer architecture and organisation
References	www.openmp.org

Date:	Setting Up Jupyter Notebook on Google Colab for CUDA programming				
04-09-2025					
Objectives	4. To set up the Jupyter Notebook on Google Colab.				
Problem Statement	This document describes how to setup the Jupyter Notebook on Google Colab that that C++ codes can be executed leveraging GPU capabilities.				
Requirements	Colab				
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores			6
	2	Clock			16 GB Tesla T4
	3	RAM			12.7 GB
	4	Hard Disk Space			112 GB
Software:	S.No	Software	Version Details	Description	
	1	Google Colab	Free Version Cloud Engine	Allows to connect to preconfigured instances on the cloud depending on your subscription type. Notebook style software that allows you to write and execute codes in real time.	
Program(s)					
Instructions for Setting Up CUDA C++ in Colab	Steps to be followed: 1. Sign up with google account at colab.research.google.com 2. Go to File and select “New Notebook in drive”. 3. Select runtime in the menu 4. Go to “Change Runtime Type” 5. Set Hardware accelerator to T4 GPU. After this locate and click on connect on the top right side of the screen. You are all setup and ready to execute.				
Install CUDA C++ plugin for Colab	<pre>!pip install nvcc4jupyter</pre> <pre>%load_ext nvcc4jupyter</pre>				

Detect selected GPU
and its NVIDA
architecture

```
import subprocess

gpu_info = subprocess.getoutput("nvidia-smi --query-gpu=name,compute_cap --format=csv,noheader,nounits")

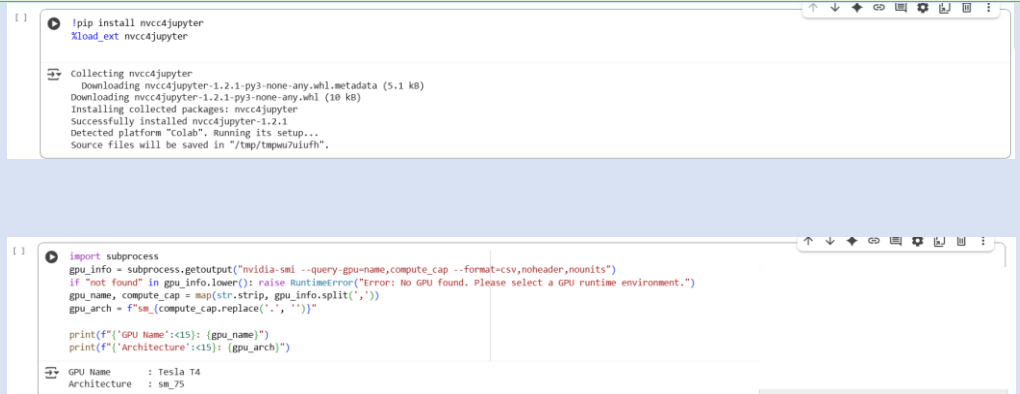
if "not found" in gpu_info.lower(): raise RuntimeError("Error: No GPU found. Please select a GPU runtime environment.")

gpu_name, compute_cap = map(str.strip, gpu_info.split(','))

gpu_arch = f'sm_{compute_cap.replace('.', '')}'

print(f'GPU Name:<15>: {gpu_name}')
print(f'Architecture:<15>: {gpu_arch}')
```

Input & Output
Snapshots:



```
[1] !pip install nvcc4jupyter
!load_ext nvcc4jupyter

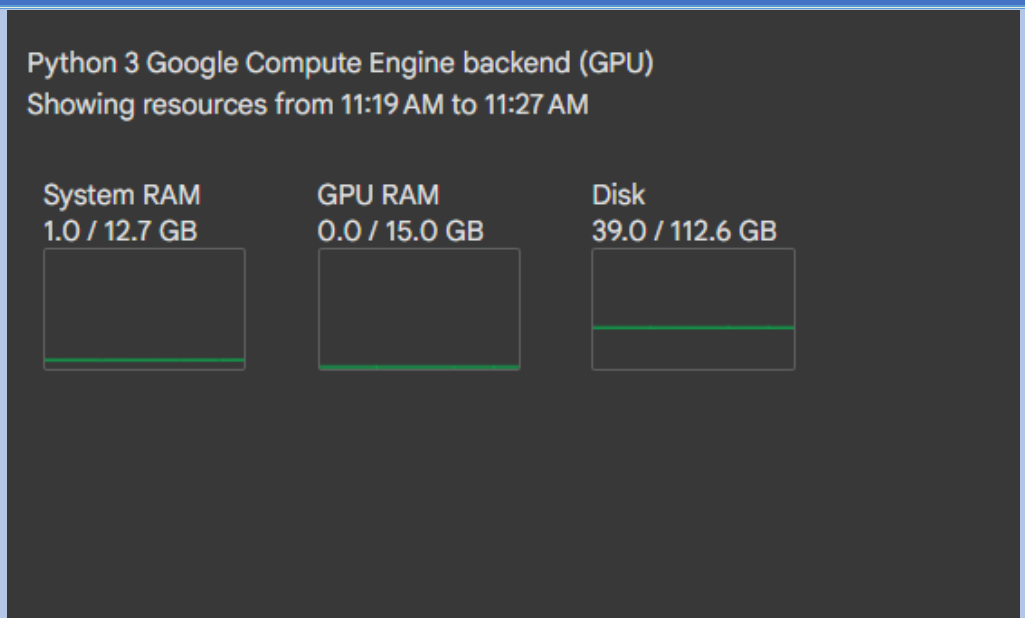
Collecting nvcc4jupyter
  Downloading nvcc4jupyter-1.2.1-py3-none-any.whl.metadata (5.1 kB)
  Downloading nvcc4jupyter-1.2.1-py3-none-any.whl (10 kB)
Installing collected packages: nvcc4jupyter
Successfully installed nvcc4jupyter-1.2.1
Detected platform "colab". Running its setup...
Source files will be saved in "/tmp/tmpwuzulufh".

[1] !import subprocess
gpu_info = subprocess.getoutput("nvidia-smi --query-gpu=name,compute_cap --format=csv,noheader,nounits")
if "not found" in gpu_info.lower(): raise RuntimeError("Error: No GPU found. Please select a GPU runtime environment.")
gpu_name, compute_cap = map(str.strip, gpu_info.split(','))
gpu_arch = f'sm_{compute_cap.replace('.', '')}'

print(f'GPU Name:<15>: {gpu_name}')
print(f'Architecture:<15>: {gpu_arch}')
```

GPU Name : Tesla T4
Architecture : sm_75

Experimental Set-
up Details



Observations	This code installs and loads the nvcc4jupyter extension, which allows running CUDA C++ code directly inside Jupyter notebooks. It then uses nvidia-smi to detect the available GPU, extracting its name and compute capability . Based on the compute capability, it constructs the corresponding NVIDIA GPU architecture code (e.g., sm_86). Finally, it prints out the GPU model and its architecture. In short: the code sets up CUDA support in Jupyter, checks for a GPU, identifies its specifications, and prepares the right architecture flag for CUDA compilation.
Source code link	https://colab.research.google.com/drive/1Py7QgAQeVO1wH0UvkfRv7ScVgG3fFoTN?usp=sharing
References	https://docs.nvidia.com/cuda/cuda-c-programming-guide/

Date: 04-09-2025	Perform Parallel Vector addition, subtraction, multiplication and division using CUDA				
Objectives	1. Parallely execute code multiple thread on device for various mathematical operations				
Problem Statement	Write a program that executes parallel execution on GPU to perform parallel execution of various mathematical operations				
Requirements	The cloud system configuration as on Colab. Subject to change as per google’s policies.				
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores			6
	2	Clock			15GB Tesla T4
	3	RAM			12.7 GB
	4	Hard Disk Space			112 GB
Software:	S.No	Software	Version Details	Description	
	1	Google Colab	Free version available on cloud Engine	Allows to connect to preconfigured instances on the cloud depending on your subscription type.	
List of functions in each file	Sl.No	Function Name		Description	
	1	void add(int *a, int *b, int *c, int n)		Adds 2 vectors and prints the sum.	
	2	void sub(int *a, int *b, int *c, int n)		Subtract one vector from other and print remainder.	
	3	void multiply(int *a, int *b, int *c, int n)		Multiplies the two vectors and prints the product.	
	4	void divide(int *a, int *b, int *c, int n)		Divide the two vectors and prints the integer quotient.	
Program(s)					
Vector addition	%%cuda -c "--gpu-architecture \$gpu_arch"				

```

#include <stdio.h>

// CUDA kernel to add two vectors
__global__ void add(int *a, int *b, int *c, int n) {
    int i = threadIdx.x;      // Thread index
    if (i < n) c[i] = a[i] + b[i]; // Each thread adds one element
}

int main() {
    int n = 5;
    int a[5] = {1,2,3,4,5};    // Host array A
    int b[5] = {10,20,30,40,50}; // Host array B
    int c[5];                  // Result array C

    int *d_a, *d_b, *d_c;      // Device arrays
    cudaMalloc(&d_a, n*sizeof(int)); // Allocate GPU memory
    cudaMalloc(&d_b, n*sizeof(int));
    cudaMalloc(&d_c, n*sizeof(int));

    // Copy data from host to device
    cudaMemcpy(d_a, a, n*sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, n*sizeof(int), cudaMemcpyHostToDevice);

    // Launch kernel with n threads
    add<<<1,n>>>>(d_a, d_b, d_c, n);

    // Copy result back to host
    cudaMemcpy(c, d_c, n*sizeof(int), cudaMemcpyDeviceToHost);

    // Print result
    for(int i=0; i<n; i++) printf("%d ", c[i]);
    printf("\n");

    // Free GPU memory

```

	<pre> cudaFree(d_a); cudaFree(d_b); cudaFree(d_c); } </pre>
Vector subtraction	<pre> %%cuda -c "--gpu-architecture \$gpu_arch" #import <stdio.h> __global__ void sub(int *a, int *b, int *c, int n) { int i = threadIdx.x; if(i < n){ if(a[i] > b[i]) c[i] = a[i] - b[i]; else c[i] = b[i] - a[i]; } } int main(){ int n = 5; int a[5] = {1,2,3,4,5}; int b[5] = {10,20,30,40,50}; int c[5]; int *d_a, *d_b, *d_c; cudaMalloc(&d_a, n*sizeof(int)); cudaMalloc(&d_b, n*sizeof(int)); cudaMalloc(&d_c, n*sizeof(int)); cudaMemcpy(d_a,a,n*sizeof(int),cudaMemcpyHostToDevice); cudaMemcpy(d_b,b,n*sizeof(int),cudaMemcpyHostToDevice); sub<<<1,n>>>>(d_a,d_b,d_c,n); cudaMemcpy(c,d_c,n*sizeof(int),cudaMemcpyDeviceToHost); </pre>

	<pre> for(int i=0;i<n;i++) printf("%d ",c[i]); printf("\n"); cudaFree(d_a); cudaFree(d_b); cudaFree(d_c); } </pre>
Vector Multiplication	<pre> %%cuda -c "--gpu-architecture \$gpu_arch" #include <stdio.h> __global__ void mul(int *a, int *b, int *c, int n){ int i = threadIdx.x; if(i<n) c[i] = a[i] * b[i]; } int main(){ int n = 5; int a[5] = {1,2,3,4,5}; int b[5] = {10,20,30,40,50}; int c[5]; int *d_a, *d_b, *d_c; cudaMalloc(&d_a, n*sizeof(int)); cudaMalloc(&d_b, n*sizeof(int)); cudaMalloc(&d_c, n*sizeof(int)); cudaMemcpy(d_a,a,n*sizeof(int),cudaMemcpyHostToDevice); cudaMemcpy(d_b,b,n*sizeof(int),cudaMemcpyHostToDevice); mul<<<1,n>>>>(d_a,d_b,d_c,n); cudaMemcpy(c,d_c,n*sizeof(int),cudaMemcpyDeviceToHost); </pre>

	<pre> for(int i=0;i<n;i++) printf("%d ",c[i]); printf("\n"); cudaFree(d_a); cudaFree(d_b); cudaFree(d_c); } </pre>
Vector Division	<pre> %%cuda -c "--gpu-architecture \$gpu_arch" #include <stdio.h> __global__ void div(int *a, int *b, int *c, int n){ int i = threadIdx.x; if(i<n){ if(b[i] != 0 && b[i]<a[i]) c[i] = a[i] / b[i]; else{ c[i] = b[i] / a[i]; } } } int main(){ int n = 5; int a[5] = {1,2,3,4,5}; int b[5] = {10,20,30,40,50}; int c[5]; int *d_a, *d_b, *d_c; cudaMalloc(&d_a, n*sizeof(int)); cudaMalloc(&d_b, n*sizeof(int)); cudaMalloc(&d_c, n*sizeof(int)); </pre>

```

cudaMemcpy(d_a,a,n*sizeof(int),cudaMemcpyHostToDevice);

cudaMemcpy(d_b,b,n*sizeof(int),cudaMemcpyHostToDevice);


div<<<1,n>>>>(d_a,d_b,d_c,n);


cudaMemcpy(c,d_c,n*sizeof(int),cudaMemcpyDeviceToHost);


for(int i=0;i<n;i++) printf("%d ",c[i]);

printf("\n");


cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
}

```



Figure 3: output of vector addition

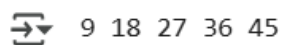


Figure 4: output of vector subtraction

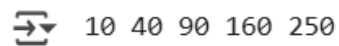


Figure 5: output of vector multiplication

Input & Output
Snapshots:

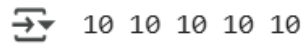
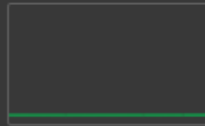
The image shows a code cell output with a double arrow icon followed by the text "10 10 10 10 10".

Figure 6: output of vector division

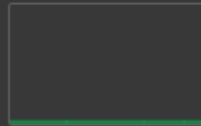
Experimental Set-up Details

Python 3 Google Compute Engine backend (GPU)
Showing resources from 11:19 AM to 11:27 AM

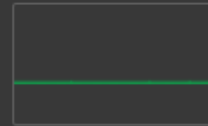
System RAM
1.0 / 12.7 GB



GPU RAM
0.0 / 15.0 GB



Disk
39.0 / 112.6 GB



Observations

1. **CUDA Kernels:** Three separate code cells define and execute CUDA C++ programs:
 - **Vector Addition:** A kernel add is defined and executed to perform element-wise addition of two integer arrays a and b, storing the result in c.
 - **Vector Subtraction:** A kernel sub is defined and executed to perform element-wise subtraction of two integer arrays a and b, storing the result in c.
 - **Vector Multiplication:** A kernel multiply is defined and executed to perform element-wise multiplication of two integer arrays.
 - **Vector Division:** A kernel divide is defined and executed to perform element-wise division of two integer arrays.
2. **Data Transfer and Execution:** In each of the CUDA code cells, memory is allocated on the GPU (cudaMalloc), data is copied from the host (CPU) to the device (GPU) (cudaMemcpy), the respective CUDA kernel is launched (<<<1, n>>>), the results are copied back from the device to the host (cudaMemcpy), and finally, the results are printed. GPU memory is freed at the end of each program.

In essence, the notebook shows how to leverage the GPU for simple parallel computations by writing and executing CUDA code directly within Colab.

Source code link	https://colab.research.google.com/drive/1Py7QgAQeVO1wH0UvkfRv7ScVgG3fFoTN?usp=sharing
References	https://docs.nvidia.com/cuda/cuda-c-programming-guide/

Week No: 10 Date:07-08-2025 S.No: 25	Real-Time Traffic Flow Analyzer using OpenMP				
Objectives:	To design and implement a parallel program that processes real-time traffic data (vehicle counts and average speeds) from multiple junctions using OpenMP . The goal is to efficiently compute per-junction statistics such as average vehicle count , average speed , and a congestion index using shared-memory parallelism.				
Problem Statement	Develop an OpenMP-based parallel system to analyze simulated real-time traffic flow data collected from 20 junctions over a 24-hour period. The program must: • Generate synthetic traffic data (vehicle count and speed per second). • Compute the following metrics for each junction: ○ Average vehicle count ○ Minimum and maximum speed ○ Congestion index = (Average vehicle count / Average speed) • Display which thread processed each junction. • Measure and display the total execution time for performance comparison.				
Requirements:					
Hardware:	S.No	Hardware	Minimum Req.	Recommended	My system
	1	Number of Cores	2	4+	6
	2	Clock	1.6 GHz	2.5 GHz	2.6 GHz
	3	RAM	4 GB	4GB or more	8 GB
	4	Hard Disk Space	10 GB free	50 + GB free	1 TB
Software:	S.No	Software	Version Details	Description	
	1	Visual studio	2022	Visual Studio is an integrated development environment (IDE) from Microsoft that supports multiple programming languages, offering code editing, debugging, and project management tools for efficient software development across platforms.	
	2	OpenMP	2.0	OpenMP 2.0 is an API that supports multi-platform shared-memory parallel programming in C, C++, and Fortran, enabling developers to write parallel code using compiler directives and runtime routines.	

List of Files Supplied:	S.No	File Name	Description	
	1	traffic_analyser.h	It includes functions to implement parallelising of analysis of weather with OpenMP	
	2	25MCS0020_main.cpp	Used to run the weather data analyser program	
List of functions in each file	S.No	File Name	Function Name	Description
	1	traffic_analyser.h	traffic_analyser()	Contains the traffic data processing functions
	2	25MAI0079_main.cpp	main()	Used to call the function
Program(s)				
	<pre>#include <stdio.h> #include <stdlib.h> #include <omp.h> #include <time.h> #define JUNCTIONS 20 #define SECONDS_PER_DAY 86400 // simulate 1 day of traffic (can be reduced for testing) void fn_traffic_analyser() { int vehicle_counts[JUNCTIONS][1000]; // sample 1000 seconds of traffic float speeds[JUNCTIONS][1000]; float avg_count[JUNCTIONS]; float avg_speed[JUNCTIONS]; float congestion_index[JUNCTIONS]; float min_speed[JUNCTIONS]; float max_speed[JUNCTIONS]; srand(time(NULL)); // Simulated traffic data</pre>			

traffic_analyser.h

```
for (int i = 0; i < JUNCTIONS; i++) {  
    for (int j = 0; j < 1000; j++) {  
        vehicle_counts[i][j] = rand() % 200;    // 0–199 vehicles per second  
        speeds[i][j] = (rand() % 80) + 20;    // 20–100 km/h  
    }  
}  
  
double start_time = omp_get_wtime();  
  
#pragma omp parallel for  
for (int i = 0; i < JUNCTIONS; i++) {  
    float sum_speed = 0.0, sum_count = 0.0;  
    float min_v = speeds[i][0];  
    float max_v = speeds[i][0];  
  
    for (int j = 0; j < 1000; j++) {  
        float spd = speeds[i][j];  
        sum_speed += spd;  
        sum_count += vehicle_counts[i][j];  
        if (spd < min_v) min_v = spd;  
        if (spd > max_v) max_v = spd;  
    }  
  
    avg_speed[i] = sum_speed / 1000.0;  
    avg_count[i] = sum_count / 1000.0;  
    congestion_index[i] = avg_count[i] / avg_speed[i];  
    min_speed[i] = min_v;  
    max_speed[i] = max_v;  
  
    printf("Junction %2d handled by Thread %d | AvgCount: %.2f | AvgSpeed: %.2f | CI: %.3f  
| MinSpd: %.2f | MaxSpd: %.2f\n",  
        i + 1, omp_get_thread_num(), avg_count[i], avg_speed[i], congestion_index[i], min_v,  
        max_v);  
}
```

	<pre> double end_time = omp_get_wtime(); printf("\nTraffic Analysis Completed in %.3f seconds using %d threads.\n", end_time - start_time, omp_get_max_threads()); } </pre>		
25MAI0079_main.cpp	<pre> #include <stdio.h> #include <omp.h> #include "traffic_analyser.h" int main() { printf("=== REAL-TIME TRAFFIC FLOW ANALYZER USING OPENMP ===\n\n"); fn_traffic_analyser(); return 0; } </pre>		
Description of OpenMP constructs	S.No	OpenMP Construct	Category of Construct & Description of construct with reference to the application/program
	1	#pragma omp parallel for	Work-Sharing Construct: Distributes loop iterations across multiple threads.
	2	omp_get_thread_num()	Runtime Routine: Identifies which thread is handling which part of the computation
	3	omp_get_wtime()	Runtime Routine: Measures execution time for performance comparison
	4	omp_get_max_threads()	Runtime Routine: Returns total number of threads available in parallel region
Input & Output Snapshots:	Start Without Debugging:		

	<pre>=== REAL-TIME TRAFFIC FLOW ANALYZER USING OPENMP === Junction 1 handled by Thread 0 AvgCount: 99.24 AvgSpeed: 60.11 CI: 1.651 MinSpd: 20.00 MaxSpd: 99.00 Junction 2 handled by Thread 0 AvgCount: 99.17 AvgSpeed: 60.18 CI: 1.648 MinSpd: 20.00 MaxSpd: 99.00 Junction 3 handled by Thread 0 AvgCount: 102.36 AvgSpeed: 60.08 CI: 1.704 MinSpd: 20.00 MaxSpd: 99.00 Junction 4 handled by Thread 0 AvgCount: 97.93 AvgSpeed: 59.34 CI: 1.650 MinSpd: 20.00 MaxSpd: 99.00 Junction 5 handled by Thread 0 AvgCount: 98.99 AvgSpeed: 59.41 CI: 1.666 MinSpd: 20.00 MaxSpd: 99.00 Junction 6 handled by Thread 0 AvgCount: 98.89 AvgSpeed: 60.38 CI: 1.638 MinSpd: 20.00 MaxSpd: 99.00 Junction 7 handled by Thread 0 AvgCount: 100.41 AvgSpeed: 59.33 CI: 1.692 MinSpd: 20.00 MaxSpd: 99.00 Junction 8 handled by Thread 0 AvgCount: 101.19 AvgSpeed: 59.97 CI: 1.688 MinSpd: 20.00 MaxSpd: 99.00 Junction 9 handled by Thread 0 AvgCount: 99.43 AvgSpeed: 59.11 CI: 1.682 MinSpd: 20.00 MaxSpd: 99.00 Junction 10 handled by Thread 0 AvgCount: 97.21 AvgSpeed: 60.65 CI: 1.603 MinSpd: 20.00 MaxSpd: 99.00 Junction 11 handled by Thread 0 AvgCount: 101.25 AvgSpeed: 60.31 CI: 1.679 MinSpd: 20.00 MaxSpd: 99.00 Junction 12 handled by Thread 0 AvgCount: 98.30 AvgSpeed: 58.71 CI: 1.674 MinSpd: 20.00 MaxSpd: 99.00 Junction 13 handled by Thread 0 AvgCount: 98.70 AvgSpeed: 59.38 CI: 1.662 MinSpd: 20.00 MaxSpd: 99.00 Junction 14 handled by Thread 0 AvgCount: 99.15 AvgSpeed: 59.56 CI: 1.665 MinSpd: 20.00 MaxSpd: 99.00 Junction 15 handled by Thread 0 AvgCount: 98.43 AvgSpeed: 60.38 CI: 1.630 MinSpd: 20.00 MaxSpd: 99.00 Junction 16 handled by Thread 0 AvgCount: 99.58 AvgSpeed: 58.81 CI: 1.693 MinSpd: 20.00 MaxSpd: 99.00 Junction 17 handled by Thread 0 AvgCount: 98.77 AvgSpeed: 58.72 CI: 1.682 MinSpd: 20.00 MaxSpd: 99.00 Junction 18 handled by Thread 0 AvgCount: 103.85 AvgSpeed: 60.67 CI: 1.712 MinSpd: 20.00 MaxSpd: 99.00 Junction 19 handled by Thread 0 AvgCount: 102.27 AvgSpeed: 60.80 CI: 1.682 MinSpd: 20.00 MaxSpd: 99.00 Junction 20 handled by Thread 0 AvgCount: 93.59 AvgSpeed: 60.65 CI: 1.543 MinSpd: 20.00 MaxSpd: 99.00 Traffic Analysis Completed in 0.002 seconds using 8 threads.</pre>																																
Experimental Set-up Details	<table><tr><td>Utilization</td><td>Speed</td><td>Base speed:</td><td>3.30 GHz</td></tr><tr><td>7%</td><td>1.28 GHz</td><td>Sockets:</td><td>1</td></tr><tr><td>Processes</td><td>Threads</td><td>Cores:</td><td>4</td></tr><tr><td>254</td><td>3855</td><td>Logical processors:</td><td>8</td></tr><tr><td>Handles</td><td></td><td>Virtualization:</td><td>Enabled</td></tr><tr><td>127324</td><td></td><td>L1 cache:</td><td>320 KB</td></tr><tr><td>Up time</td><td></td><td>L2 cache:</td><td>5.0 MB</td></tr><tr><td>10:01:00:28</td><td></td><td>L3 cache:</td><td>12.0 MB</td></tr></table>	Utilization	Speed	Base speed:	3.30 GHz	7%	1.28 GHz	Sockets:	1	Processes	Threads	Cores:	4	254	3855	Logical processors:	8	Handles		Virtualization:	Enabled	127324		L1 cache:	320 KB	Up time		L2 cache:	5.0 MB	10:01:00:28		L3 cache:	12.0 MB
Utilization	Speed	Base speed:	3.30 GHz																														
7%	1.28 GHz	Sockets:	1																														
Processes	Threads	Cores:	4																														
254	3855	Logical processors:	8																														
Handles		Virtualization:	Enabled																														
127324		L1 cache:	320 KB																														
Up time		L2 cache:	5.0 MB																														
10:01:00:28		L3 cache:	12.0 MB																														
Observations	xv. The program efficiently parallelizes per-junction data analysis. xvi. Independent nature of each junction ensures excellent scalability. xvii. Execution time decreases significantly with more threads. xviii. Ideal model for smart city real-time traffic monitoring systems.																																
Source code link	Richie21r/25mai0079_CAO: course work :computer architecture and organisation																																
References	www.openmp.org																																